

Wykład 9

Uczenie ze wzmocnieniem

Uczenie przez wzmacnianie (uczenie posiłkowane)

RL - *Reinforcement learning*

Czym jest uczenie ze wzmocnieniem RL

Definicja

Uczenie przez doświadczenia/dane w celu podejmowania właściwych decyzji w warunkach niepewności.

Czym jest uczenie ze wzmocnieniem RL

Definicja (Wikipedia)

Jeden z trzech głównych nurtów uczenia maszynowego, którego zadaniem jest interakcja ze środowiskiem za pomocą polityki/strategii na podstawie zebranych informacji. W uczeniu przez wzmacnianie nie przygotowuje się zestawu danych uczących, tylko środowisko (ang. *environment*), z którego model będzie zbierał dane automatycznie; jego celem jest **zmaksymalizowanie zwracanej przez nie nagrody**.

Środowisko jest dynamiczne, zmieniające się.

Czym jest uczenie ze wzmocnieniem RL

W uczeniu się przez wzmocnianie agent oprogramowania (**software agent**) to inteligentny program komputerowy, który uczy się podejmować decyzje w dynamicznym środowisku, aby zmaksymalizować oczekiwaną nagrodę. Agent wchodzi w interakcję ze swoim otoczeniem, otrzymuje informacje zwrotne (nagrody lub kary) i aktualizuje swoją wewnętrzną politykę na podstawie tych doświadczeń. Ten proces prób i błędów pozwala agentowi nauczyć się optymalnych strategii osiągnięcia określonego celu.

Algorytm, wykorzystywany przez agenta nazywa się jego strategią/polityką

Geneza

Termin „wzmocnienie” w kontekście uczenia się zwierząt po raz pierwszy pojawił się w angielskim tłumaczeniu monografii Pawłowa z 1927 r. na temat odruchów warunkowych. Pawłow opisał wzmocnienie jako wzmocnienie wzorca zachowania spowodowane otrzymaniem przez zwierzę bodźca – wzmacniacza.

Pomysł wdrożenia uczenia się metodą prób i błędów w komputerze pojawił się bardzo wcześnie. Początki rozważań nad uczeniem metodą prób i błędów pochodzą z 1850 lat.

W raporcie z 1948 roku Alan Turing opisał projekt „systemu przyjemności i bólu”, który działał zgodnie z prawem efektu.

Geneza

Skonstruowano wiele pomysłowych maszyn elektromechanicznych, które działały metodą uczenia się metodą prób i błędów.

Najwcześniejszą z nich była maszyna zbudowana przez Thomasa Rossa (1933), która była w stanie znaleźć drogę przez prosty labirynt i zapamiętać ścieżkę poprzez ustawienia przełączników. W 1951 roku W. Grey Walter zbudował wersję swojego „mechanicznego żółwia” (Walter, 1950) zdolną do prostej formy uczenia się.

W 1952 r. Claude Shannon zademonstrował biegającą po labiryncie mysz o imieniu Theseus, która metodą prób i błędów znajdowała drogę przez labirynt, przy czym sam labirynt zapamiętywał pomyślne kierunki za pomocą magnesów i przekaźników pod podłogą (zob. także Shannon, 1951). J. A. Deutsch (1954) opisał maszynę do rozwiązywania labiryntów opartą na jego teorii zachowania (Deutsch, 1953), która ma pewne cechy wspólne z uczeniem się ze wzmocnieniem opartym na modelu. Była maszyna do grania w O i X i wiele innych osiągnięć.

Typy problemów uczenia maszynowego

	Uczenie nadzorowane	Uczenie nienadzorowane	Uczenie ze wzmocnieniem
Dane	(x, y) x-cechy, y-etykiety	x-dane, brak etykiet	pary (stan, akcja)
Cel	Znalezienie funkcji, która przekształca x w y	Wykrycie wzorców	Maksymalizacja oczekiwanej przyszłej nagrody
Przykład	To jest uśmiechnięta twarz 	 te kształty są podobne	Uśmiechnij się, bo to poprawia wygląd 
	Klasyfikacja Regresja (predykcja)	Grupowanie (m.in. klasteryzacja, analiza skupień). Redukcja wymiarów. Uzupełnianie wartości.	Gry komputerowe, samochody autonomiczne, roboty

Uczenie ze wzmocnieniem

Co odróżnia uczenie ze wzmocnieniem od innych działów uczenia maszynowego:

- Nie potrzebna jest baza danych - agent uczy się na podstawie informacji o nagrodach otrzymywanych ze środowiska.
- Nagroda może być odłożona w czasie.
- Czas ma znaczenie.
- Działanie agenta ma wpływ na dane, jakie otrzymuje ze środowiska.

Deep Reinforcement Learning that Matters

**Peter Henderson^{1*}, Riashat Islam^{1,2*}, Philip Bachman²
Joelle Pineau¹, Doina Precup¹, David Meger¹**

¹ McGill University, Montreal, Canada

² Microsoft Maluuba, Montreal, Canada

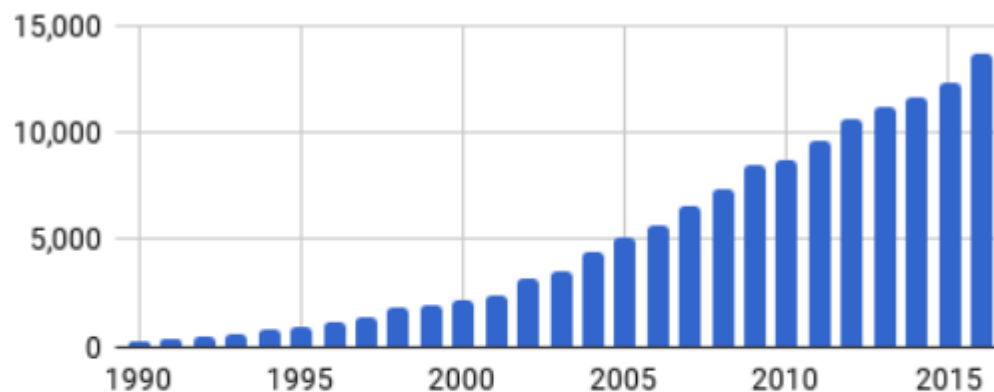


Figure 1: Growth of published reinforcement learning papers. Shown are the number of RL-related publications (y-axis) per year (x-axis) scraped from Google Scholar searches.

Polecane materiały

RL Course by David Silver –(DeepMind) i nowsze

<https://www.youtube.com/watch?v=2pWv7GOvuf0&list=PLzuuYNsE1EZAXYR4FJ75jcJseBmo4KQ9>

- CS 188: Artificial Intelligence by Pieter Abbeel (wykład 10 i 11)-

<https://www.youtube.com/watch?v=IXuHxkpO5E8> (stare)

- Practical RL Course by Yandex School of Data Analysis -

<https://github.com/yandexdataschool/PracticalRL>

- CS 234 Reinforcement Learning Stanford

<https://www.youtube.com/watch?v=WsvFL-LjA6U> (nowe)

- Reinforcement Learning from scratch Graphics in 5 Minutes

Steve Seitz <https://www.youtube.com/watch?v=vXtfdGphr3c&t=32s>

- Hands-On Machine Learning with Scikit-Learn & TensorFlow, Rozdział 16

- MIT 6.S191: Reinforcement Learning

<https://www.youtube.com/watch?v=to-lHJfK4pw>

CS 285 Sergey Levine UC Berkeley:

<https://www.youtube.com/watch?v=jds0Wh9jTvE> oraz

<https://rail.eecs.berkeley.edu/deeprlcourse/>



Richard S. Sutton and Andrew G. Barto, Reinforcement Learning: An Introduction



<https://www.youtube.com/watch?v=g6N3EnpAkJA&list=PL7SYVykTNxXbu7EZTleyrJUNMwbg37WG3>



Lecture RL1 - Reinforcement

Gerstner Lab - 4/12



RL1.1 - Examples of Reinforcement Learning

Gerstner Lab

Elements of Reinforcement Learning

RL1.2 - Elements of Reinforcement Learning

Shiyu Zhao's Open Course

Mathematical Foundations
of
Reinforcement Learning

<https://github.com/MathFoundationRL/Book-Mathematical-Foundation-of-Reinforcement-Learning>

Mathematical
Foundations of
Reinforcement
Learning

Shiyu Zhao

Strona Koła Naukowego Main

Instytut Informatyki Stosowanej
Politechnika Łódzka

Reinforcement Learning - sztuka metody prób i błędów



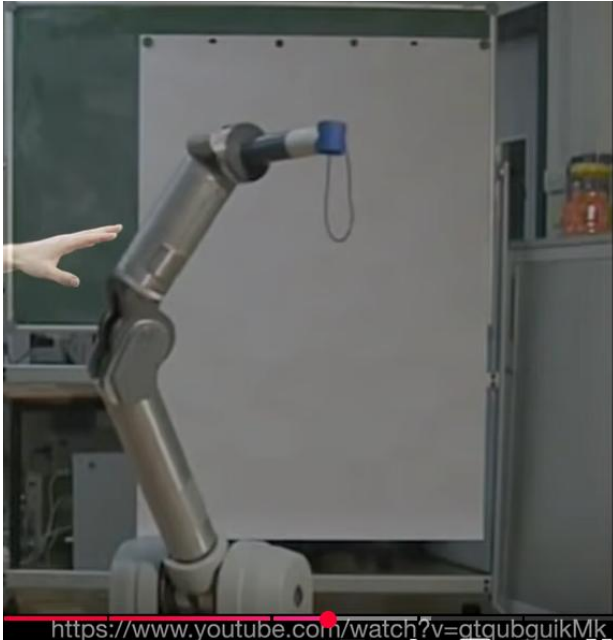
Reinforcement
learning

<https://main.p.lodz.pl/news.php?id=105>

Przykłady zastosowania



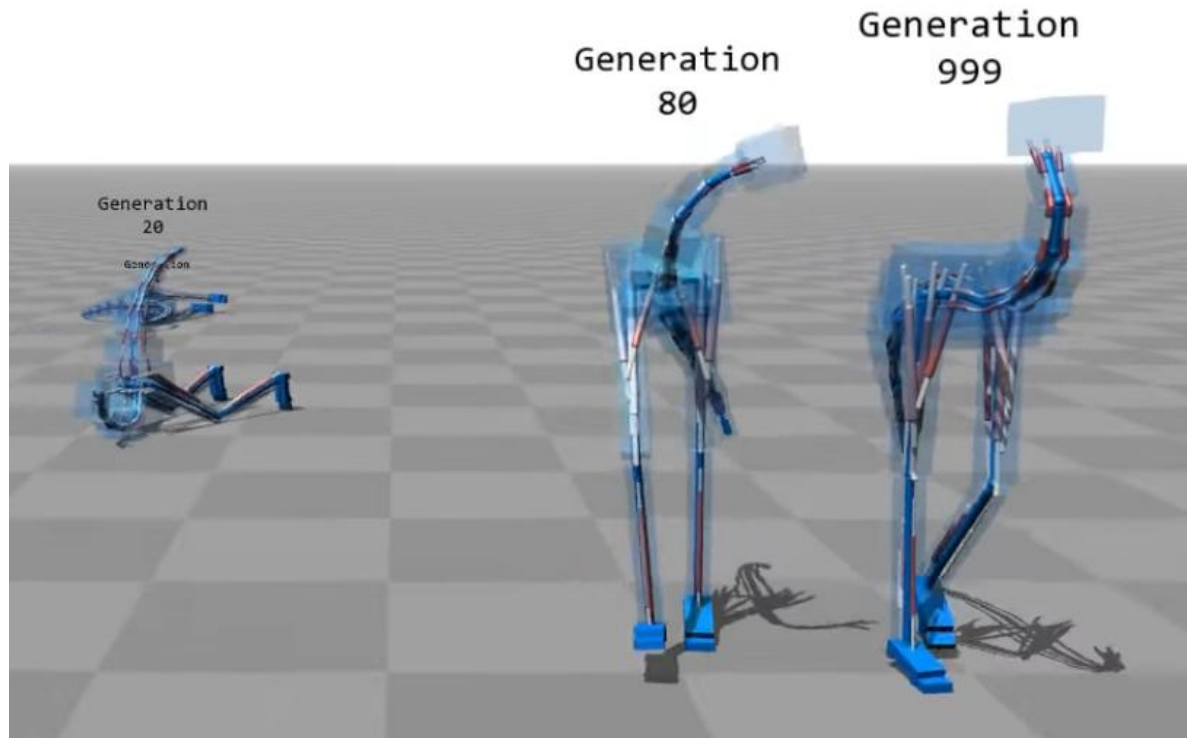
Robotyka



<https://www.youtube.com/watch?v=gtqubguikMk>

Human Demonstration for Imitation Learning

Kober, J.; Peters, J.; Learning Motor Primitives in Robotics



<https://www.youtube.com/watch?v=pgaEE27nsQw>

Flexible Muscle-Based Locomotion for Bipedal Creatures



Goatstream

5,01 tys. subskrybentów

Subskrybuj

Playing Atari with Deep Reinforcement Learning

Volodymyr Mnih Koray Kavukcuoglu David Silver Alex Graves Ioannis Antonoglou

Daan Wierstra Martin Riedmiller

DeepMind Technologies

2013

{vlad,koray,david,alex.graves,ioannis,daan,martin.riedmiller} @ deepmind.com

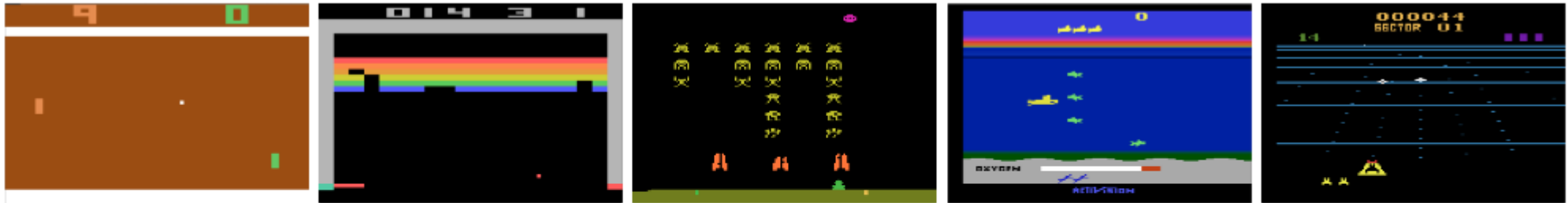


Figure 1: Screen shots from five Atari 2600 Games: (*Left-to-right*) Pong, Breakout, Space Invaders, Seaquest, Beam Rider

AlphaGo

AlphaGo mastered the ancient game of Go, defeated a Go world champion, and inspired a new era of AI systems.

Google DeepMind

2015 r.



Google DeepMind

<https://deepmind.google> › [blog](#) · [Tłumaczenie strony](#) ⋮

AlphaStar: Mastering the real-time strategy game StarCraft II

Cel: Wygranie gry

Stan: Pozycja wszystkich pionków

Akcja: Położenie następnego pionka

Nagroda: 1 jeśli gra wygrana, 0 w przeciwnym wypadku



www.deepmind.com/research/alphago/

ALPHAGO_LEE. TEORIA POŚWIĘCENIA

Polityczny thriller, teatralne „Black Mirror” z kultowym meczem Go w tle. To starcie koreańskiego mistrza – reprezentanta ludzkości – z programem AlphaGo, stworzonym przez Google DeepMind. Widowiskowa walka o panowanie nad światem, rozegrana w hotelu Four Seasons w Seulu, z udziałem performerów i artystów z Warszawy, Seulu, Londynu i Nowego Jorku.

CZAS TRWANIA 120 MINUT

SCENA DUŻA IM. JÓZEFA SZAJNY

PREMIERA 27.09.2025

**UWAGA! SPEKTAKL GRANY W JĘZYKU POLSKIM I ANGIELSKIM.
WSZYSTKIE TERMINY Z NAPISAMI W OBU JĘZYKACH.**

PRZERWA (BEZ PRZERWY)

**BILETY NORMALNY 120 ZŁ / ULGOWY 80 ZŁ / WEJŚCIÓWKA 50
ZŁ**

DOSTĘPNOŚĆ

ARCHITEKTONICZNA

**OGRANICZENIA 15+, DYM, GŁOŚNA MUZYKA, INTENSYWNE
MIGAJĄCE ŚWIATŁA, ALKOHOL**

**2. PIĘTRO, WINDA, MIEJSCE DLA
WÓZKÓW**

- <https://blog.google/intl/pl-pl/10-lat-alphago-od-legendarnego-ruchu-37-do-nowej-ery-odkryc-naukowych/>



10 lat AlphaGo: od legendarnego „Ruchu 37” do nowej ery odkryć naukowych

10 mar, 2026
4 min czytania

Od legendarnego zwycięstwa nad mistrzem świata w Go po Nagrodę Nobla – poznaj drogę od AlphaGo do współczesnych przełomów w AI.

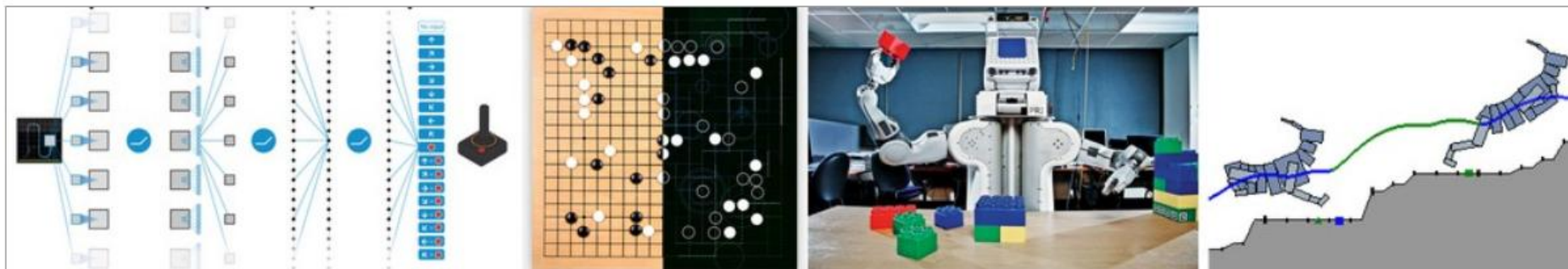


Demis Hassabis
CEO and Co-Founder, Google DeepMind

 Udostępnij



Andrej Karpathy blog



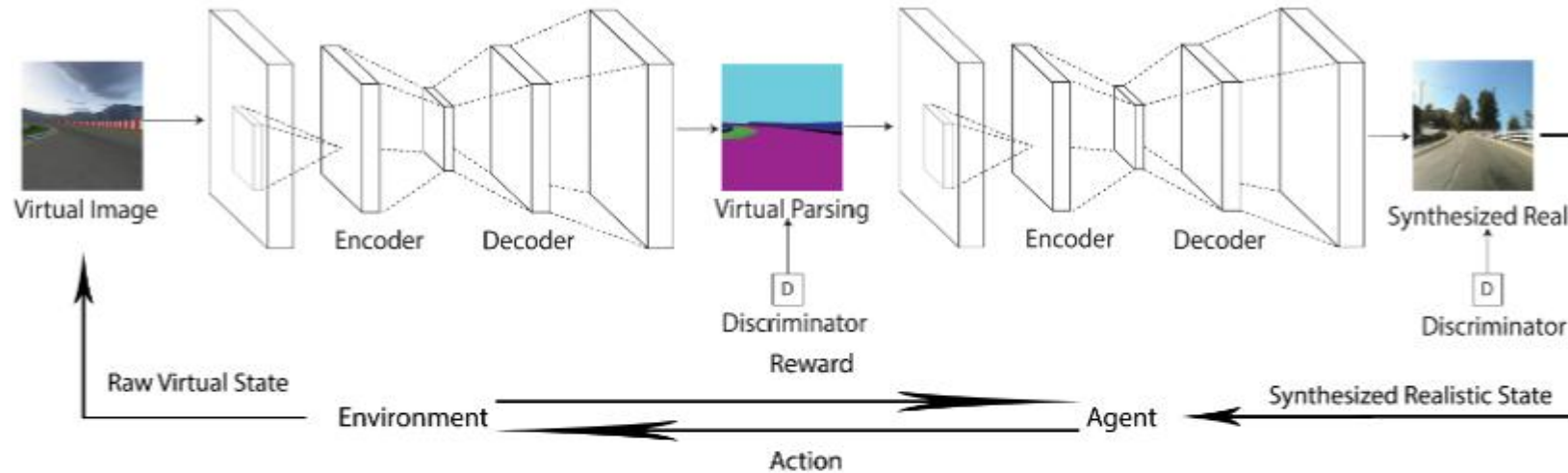
Examples of RL in the wild. From left to right: Deep Q Learning network playing ATARI, AlphaGo, Berkeley robot stacking Legos, physically-simulated quadruped leaping over terrain.

<http://karpathy.github.io/2016/05/31/rl/>

Deep Reinforcement Learning: Pong from Pixels

May 31, 2016

Virtual to Real Reinforcement Learning for Autonomous Driving

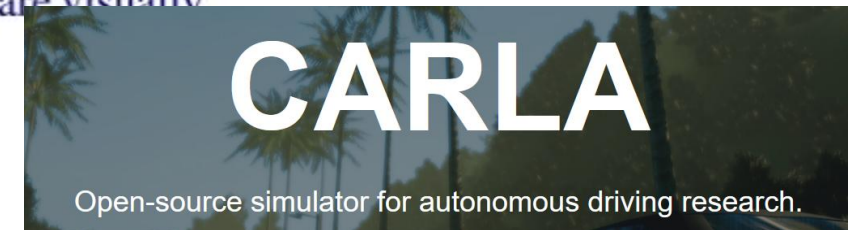


¹ University of California, Berkeley
Berkeley, CA, USA

² Shanghai Jiao Tong University
Shanghai, China

³ Tsinghua University
Beijing, China

Figure 1: Framework for virtual to real reinforcement learning for autonomous driving. Virtual images rendered by a simulator (environment) are first segmented to scene parsing representation and then translated to synthetic realistic images by the proposed image translation network (VISRI). Agent observes synthetic realistic images and takes actions. Environment will give reward to the agent. Since the agent is trained using realistic images that are visually similar to real world scenes, it can nicely adapt to real world driving.



17 May 2022



Learning to Drive in a Day

Alex Kendall Jeffrey Hawke David Janz Przemysław Mazur Daniele Reda
John-Mark Allen Vinh-Dieu Lam Alex Bewley Amar Shah

Samochody autonomiczne, inteligentne roboty, autonomiczne maszyny i bezzałogowe drony działają w oparciu o technologię Edge Computing.

Komputery brzegowe to specjalnie skonstruowane komputery przeznaczone do pracy w pobliżu źródła danych - to znaczy, że nie są one zależne od zdalnej mocy obliczeniowej.

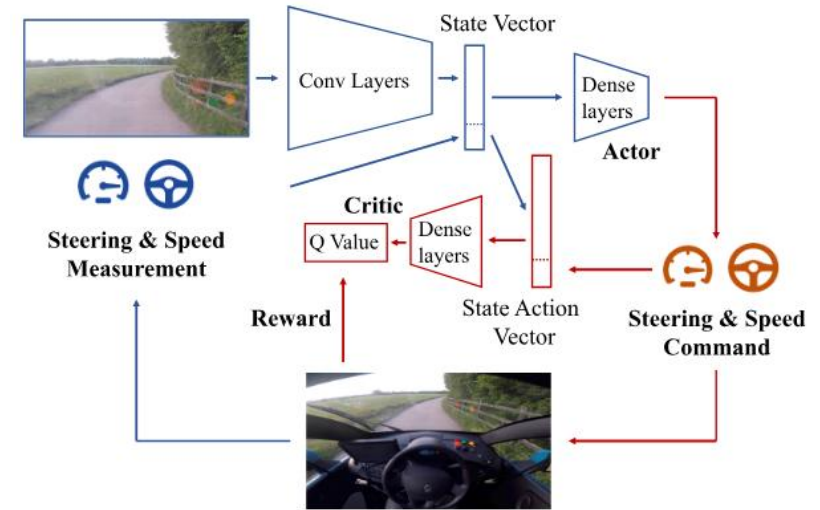


Fig. 1: We design a deep reinforcement learning algorithm for autonomous driving. This figure illustrates the actor-critic algorithm which we use to learn a policy and value function for driving. Our agent maximises the reward of distance travelled before intervention by a safety driver.

Autonomiczne auta jako przykład IoT i Edge computing

Key Points

It is an ML tool

Learns from experience of interacting

Receive feedback in the form of rewards

Learn a strategy that maximizes the **cumulative reward over time**

Components:

1. Agent - a Car in Carla Sim
2. Environment - a Town in Carla Sim
3. Actions - what you need to learn, commonly steering & throttle input
4. Observations - latest image from camera after action has been applied
5. Rewards - feedback the agent gets from Environment after each step
6. Policy - the overall strategy/model learned.

<https://www.youtube.com/watch?v=JNrUkwUGWKg>

RL

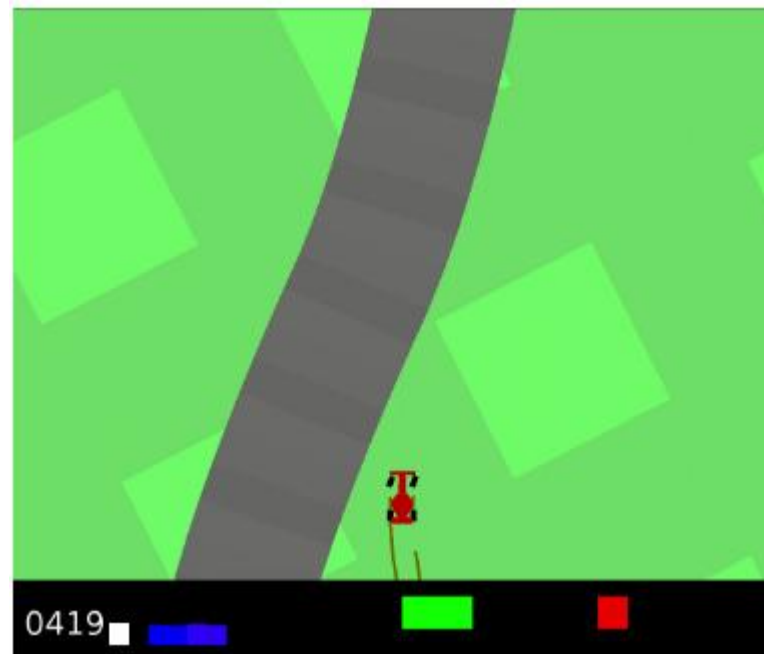
Przykład: Self-Driving Car

Cel: Podążanie pasem ruchu

Stan: Obraz (96x96)

Akcja: Przyspieszenie, kierowanie

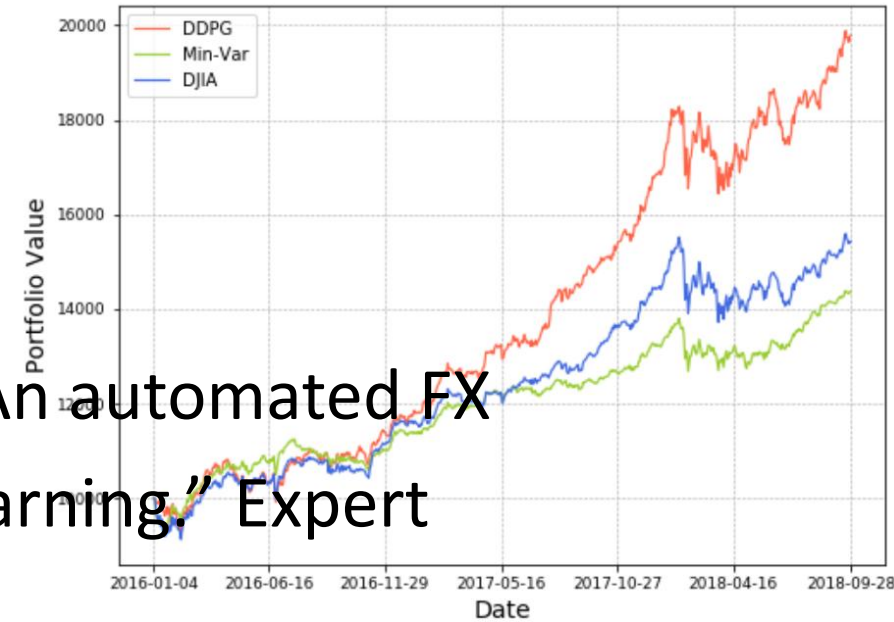
Nagroda: - za klatkę, + za kafelek



<https://gym.openai.com/envs/CarRacing-v0/>

Giełda

- Dempster, Michael AH, and Vasco Leemans. "An automated FX trading system using adaptive reinforcement learning." Expert Systems with Applications 30.3 (2006): 543-552.
- Xiong, Zhuoran, et al. "Practical deep reinforcement learning approach for stock trading." arXiv preprint arXiv:1811.07522 (2018).
- Carapuco, Joao, Rui Neves, and Nuno Horta. "Reinforcement learning applied to Forex trading." Applied Soft Computing 73 (2018):783-794.



ChatGPT (<https://openai.com/blog/chatgpt/>)

Step 1

Collect demonstration data and train a supervised policy.

A prompt is sampled from our prompt dataset.



A labeler demonstrates the desired output behavior.



This data is used to fine-tune GPT-3.5 with supervised learning.



Step 2

Collect comparison data and train a reward model.

A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.



This data is used to train our reward model.



Step 3

Optimize a policy against the reward model using the PPO reinforcement learning algorithm.

A new prompt is sampled from the dataset.



The PPO model is initialized from the supervised policy.



The policy generates an output.



The reward model calculates a reward for the output.



The reward is used to update the policy using PPO.



Proximal policy optimization (**PPO**) is a reinforcement learning (RL) algorithm for training an intelligent agent.

Definicje



Agent-podejmuje decyzje/akcje w środowisku



a_t –akcja-działanie/ruch podejmowane przez agenta w środowisku



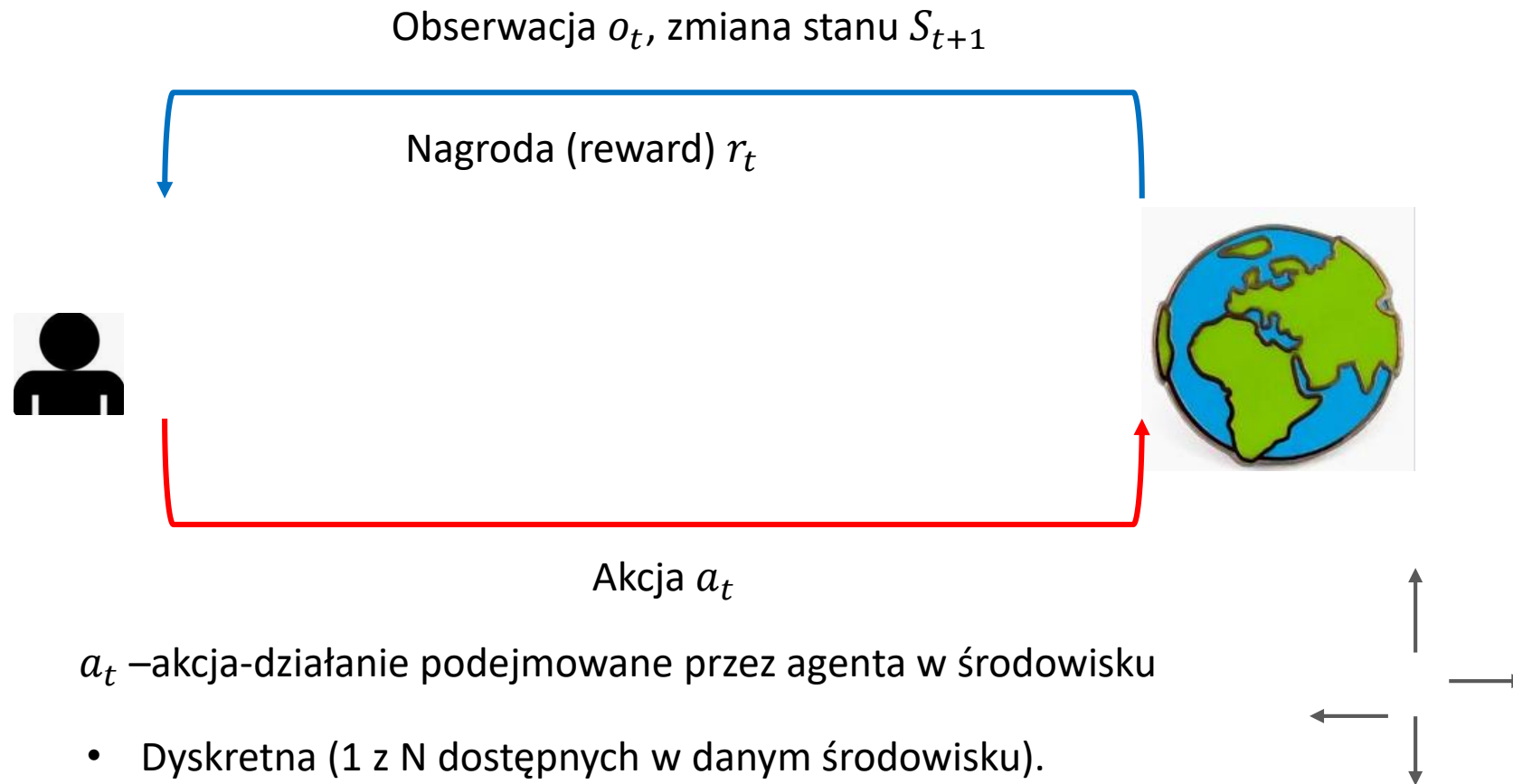
Przestrzeń akcji A: zbiór wszystkich możliwych działań, jakie agent może podjąć w środowisku.



Środowisko (ang. Environment, np. plansza do gry):
Opisuje świat, z którym agent wchodzi w interakcję

Agent wchodzi w interakcję ze środowiskiem, w którym chce osiągnąć określony cel.
Akcje podjęte przez agenta są oceniane przez środowisko.
W wyniku wykonania wybranej akcji, agent otrzymuje nagrodę (może być pozytywna lub negatywna).

Definicje



a_t –akcja-działanie podejmowane przez agenta w środowisku

- Dyskretna (1 z N dostępnych w danym środowisku).
- Ciągła (wartość lub wektor wartości)

- Agent: Osoba podejmująca decyzje (decision maker), która wykonuje działania.
- Środowisko: Świat lub system, w którym działa agent.
- Stan: Sytuacja lub stan, w którym aktualnie znajduje się agent.
- Akcja: Możliwe ruchy lub decyzje, które może podjąć agent.
- Nagroda: Informacja zwrotna lub wynik z otoczenia oparty na działaniu agenta.

<https://www.geeksforgeeks.org/what-is-reinforcement-learning/>

Podstawowe pojęcia

- Środowisko (ang. Environment, np. plansza do gry):

Opisuje świat, z którym agent wchodzi w interakcję

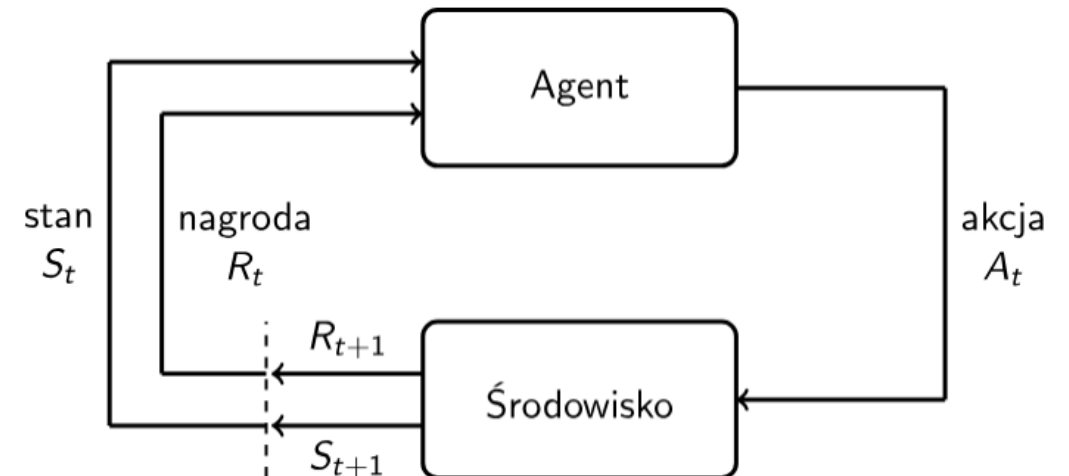
- Wejście:

Akcja

- Wyjście:

Stan

Nagroda



Model środowiska: Reprezentacja środowiska, która przewiduje przyszłe stany i nagrody, pomagając w planowaniu.

Podstawowe pojęcia

- Strategia (Polityka, Policy)

Zasady, strategia, której agent używa do określania następnej akcji na podstawie bieżącego stanu.

Może być deterministyczna lub stochastyczna.

Oznaczenie: π .

- Stan (ang. State):

Zbiór wartości opisujących aktualną sytuację.

Jest podstawą wyboru akcji przez agenta zgodnie z jego strategią.

(sytuacja, w której agent się znajduje)

Podstawowe pojęcia

- Nagroda (ang. Reward):

Wartość zwracana przez środowisko w momencie wykonania akcji wybranej przez agenta.

Reprezentuje cel, lub cele, jakie agent ma osiągnąć.

Oznaczenie: r_t - nagroda natychmiastowa otrzymana w chwili czasu t .

Funkcja nagrody:

funkcja, która zapewnia informacje zwrotne na temat wartości wynikających z podjętych akcji, kierując agenta w stronę jego celu.

Podstawowe pojęcia

- Oczekiwana nagroda (ang. Return):

Oczekiwana nagroda po zakończeniu bieżącego epizodu.

Celem uczenia za wzmocnieniem jest maksymalizacja oczekiwanej nagrody.

Oznaczenie:

Oczekiwana przyszła nagroda w chwili t :

$$G_t = R_{t+1} + R_{t+2} + \dots = \sum_{i=t+1}^{\infty} R_i$$

Zdyskontowana oczekiwana nagroda:

$$R_t = \sum_{i=t+1}^{\infty} \gamma^i R_i, \text{ gdzie } \gamma \in [0,1), \text{ discount factor, czynnik dyskontujący}$$

γ jest bardzo ważny. Odgrywa rolę kary za długość trajektorii dojścia do celu.

Przykłady ćw. 6

Podstawowe pojęcia

- Funkcja wartości (oceny, ang. Value function):

Określa, jak dobrze być w danym stanie.

Szacuje przyszłe skumulowane nagrody, które agent otrzyma z danego stanu.

Oczekiwana suma nagród, jaką możemy otrzymać, rozpoczynając w stanie s i działając zgodnie ze strategią π .

- Rodzaje:

Funkcja wartości stanu (wartość stanu, state value)

$V^\pi(s)$ - jak dobrze być w stanie s , działając zgodnie ze strategią π .

Funkcja wartości stanu-akcji (wartość akcji, action value)

$Q^\pi(s, a)$ - jak dobrze będąc w stanie s jest wykonać akcję a , działając zgodnie ze strategią π , q-value

Funkcja wartości stanu-akcji (Funkcja Q)

$$Q(s_t, a_t) = E[R_t | s_t, a_t]$$

Strategia polega na wyborze akcji, która maksymalizuje oczekiwaną nagrodę (przyszłą nagrodę)

$$\pi^*(s) = \underset{a}{\operatorname{arg\,max}} Q(s, a)$$

Podstawowe metody

Uczenie pasywne:

- Ocena strategii (ang. Policy Evaluation)
- Polepszanie strategii (ang. Policy Improvement)
- Iteracyjne doskonalenie strategii (ang. Policy Iteration)
- Iteracyjne obliczanie funkcji wartości (ang. Value Iteration)

Podstawowe metody

Uczenie aktywne:

- Metody różnic czasowych (ang. Temporal Difference Learning)
- Monte Carlo
- **Q-Learning**
- SARSA
- Metody aproksymacyjne
- Aproksymacja funkcji wartości (ang. Approximate Q-Learning)

Metody wykorzystujące głębokie sieci neuronowe

- **Deep Q-Learning**
- Double Q-Learning
- **Actor-Critic**
- REINFORCE
- **Policy Gradient**

Głębokie uczenie ze wzmocnieniem

Q-learning

Deep Q-Networks (DQNs)

Szukanie $Q(s, a)$

$$a = \underset{a}{\operatorname{argmax}} Q(s, a)$$

Policy-learning

Szukanie $\pi(s)$

Metody policy gradient w uczeniu się przez wzmocnianie bezpośrednio optymalizują strategię agenta w celu maksymalizacji oczekiwanych nagród

Q-learning to algorytm uczenia się przez wzmocnianie bez modelu, używany do trenowania agentów (programów komputerowych) w celu podejmowania optymalnych decyzji poprzez interakcję ze środowiskiem.

Przykłady

Gra na Atari z głębokim uczeniem ze wzmocnieniem, Mnih et al. , 2013
Q-learning z wykorzystaniem konwolucyjnych sieci neuronowych

Guided policy search (model-based RL) dla sterowania robotami opartego na wizji

Uczenie chodzenia z policy gradient
High-dimensional continuous control with
generalized advantage estimation, Schulman et al. '16
Trust region policy optimization with value function approximation

Uczenie chwytania przy pomocy Q-functions QT-Opt, Kalashnikov et al. '18



Przykłady zastosowania RL

- Robotyka: RL jest wykorzystywane do automatyzacji zadań w ustrukturyzowanych środowiskach, takich jak produkcja, gdzie roboty uczą się optymalizować ruchy i poprawiać wydajność.
- Gry: Zaawansowane algorytmy RL zostały wykorzystane do opracowania strategii dla złożonych gier, takich jak szachy, Go i gry wideo, w wielu przypadkach przewyższając ludzkich graczy.
- Kontrola przemysłowa: RL pomaga w dostosowywaniu i optymalizacji operacji przemysłowych w czasie rzeczywistym, takich jak procesy rafinacji w przemyśle naftowym i gazowym.
- Spersonalizowane systemy szkoleniowe: RL umożliwia dostosowanie treści instruktażowych w oparciu o indywidualne wzorce uczenia się, zwiększając zaangażowanie i skuteczność.

Breakthrough models AlphaProof and AlphaGeometry 2 solve advanced reasoning problems in mathematics

Artificial general intelligence (AGI) with advanced mathematical reasoning has the potential to unlock new frontiers in science and technology.

We've made great progress building AI systems that help mathematicians discover [new insights](#), [novel algorithms](#) and answers to [open problems](#). But current AI systems

AlphaProof: a formal approach to reasoning

AlphaProof is a system that trains itself to prove mathematical statements in the formal language [Lean](#). It couples a pre-trained language model with the [AlphaZero](#) reinforcement learning algorithm, which previously taught itself how to master the games of chess, shogi and Go.

<https://deepmind.google/discover/blog/ai-solves-imo-problems-at-silver-medal-level/>

RESEARCH

Discovering novel algorithms with AlphaTensor

5 OCTOBER 2022

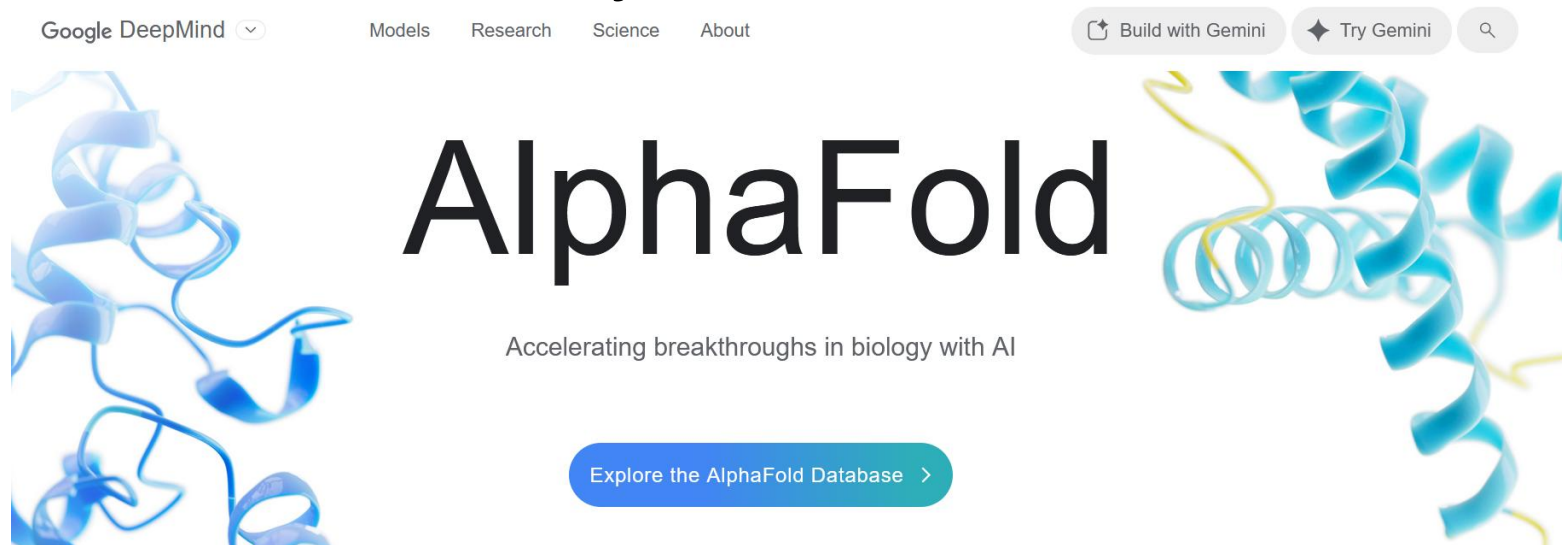
Alhussein Fawzi, Matej Balog, Bernardino Romera-Paredes, Demis Hassabis, Pushmeet Kohli

We then trained an AlphaTensor agent using reinforcement learning to play the game, starting without any knowledge about existing matrix multiplication algorithms.

Through learning, AlphaTensor gradually improves over time, re-discovering historical fast matrix multiplication algorithms such as Strassen's, eventually surpassing the realm of human intuition and discovering algorithms faster than previously known.

AlphaFold

AlphaFold system sztucznej inteligencji opracowany przez Google DeepMind, który przewiduje trójwymiarową strukturę białek na podstawie ich sekwencji aminokwasów.



[Demis Hassabis – Wikipedia, wolna encyklopedia](#)

Demis Hassabis (ur. 27 lipca 1976) – **brytyjski informatyk, badacz sztucznej inteligencji i laureat Nagrody Nobla w dziedzinie chemii za rok 2024.**

Santa Fe Institute workshop: AI and barrier of meaning 2



NEW YORK UNIVERSITY

Noyce Conference R...

Towards Machines that can Understand, Reason & Plan

Yann LeCun

Courant Institute & Center for Data Science, NYU

Meta – Fundamental AI Research

Santa Fe Institute
2023-04-24



https://www.youtube.com/watch?v=_JfEScYyVCE



Google Scholar

<https://scholar.google.com/citati...> · [Tłumaczenie strony](#) ⋮

Yann LeCun

Chief AI Scientist at Facebook & JT Schwarz Professor at the Courant Institute, New York University.

Verified email at cs.nyu.edu - Homepage.

https://pl.wikipedia.org/wiki/Yann_LeCun



AI matematycznie jest problemem nierozwiązalnym w przestrzeni | prof. Yann LeCun (Zarząd Meta)



This Is IT - Maciej Kawecki
242 tys. subskrybentów

Subskrybuj

👍 2,6 tys.



Udostępnij



Podziękuj



<https://www.youtube.com/watch?v=w-TtKPRuXL8>

Najważniejsze biblioteki do RL

- [1. TensorFlow Agents](#)
- [2. OpenAI Gym](#)
- [3. Stable Baselines3](#)
- [4. Ray RLlib](#)
- [5. Keras-RL](#)
- [6. PyTorch RL](#)
- [7. Coach](#)

Unit-tests on Linux failing Documentation Benchmarks codecov 90% Follow @torchrl1 python 3.10 | 3.11 | 3.12 | 3.13
license MIT pypi v0.10.0 nightly v2025.9.1 Downloads 1M Downloads (nightly) 3M Discord Shield

TorchRL



OpenAI Gym

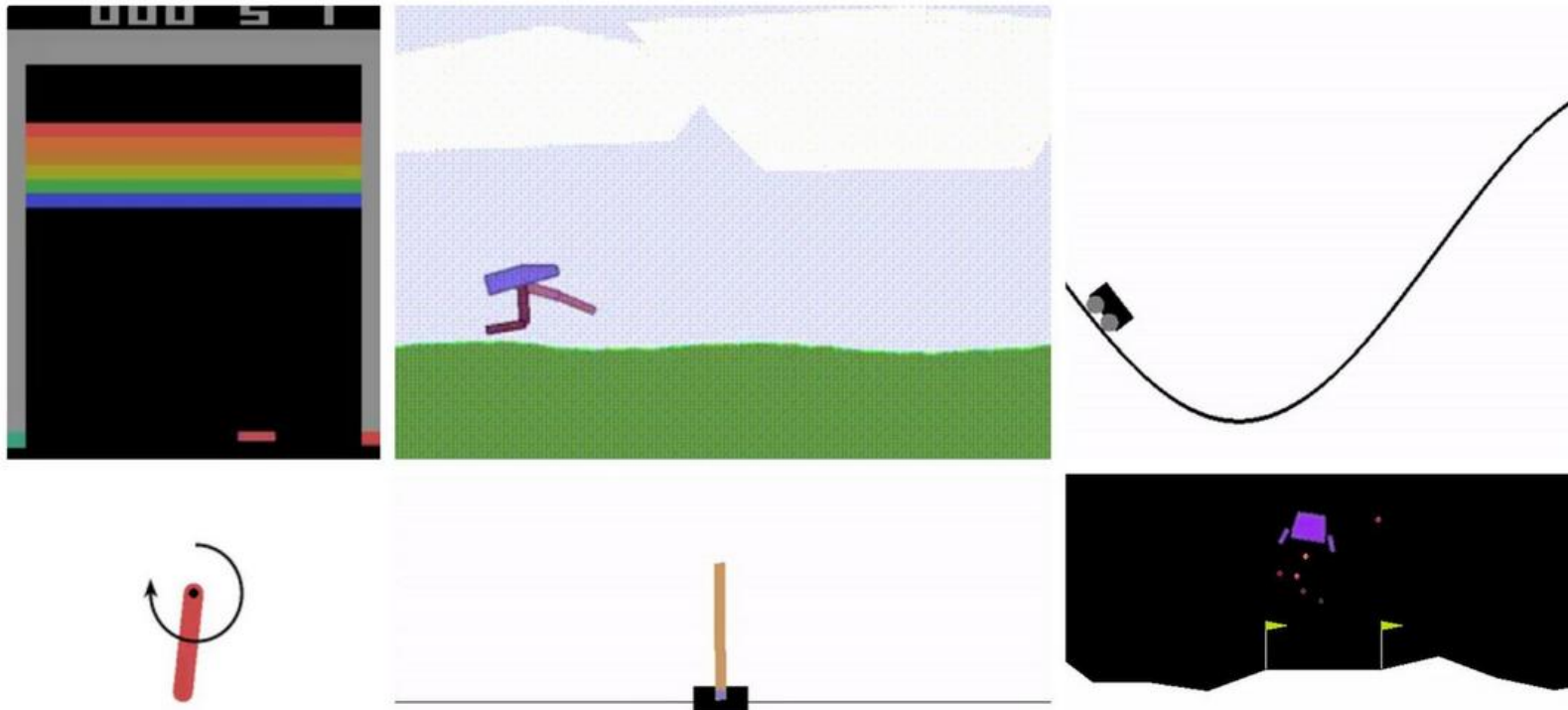
Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, Wojciech Zaremba

<https://www.geeksforgeeks.org/top-7-python-libraries-for-reinforcement-learning/>

Introduction of OpenAI Gym 🦹

OpenAI Gym is a **toolkit designed for developing and comparing reinforcement learning algorithms**. It provides a wide range of environments where researchers ...

- <https://www.digitalocean.com/community/tutorials/getting-started-with-openai-gym>

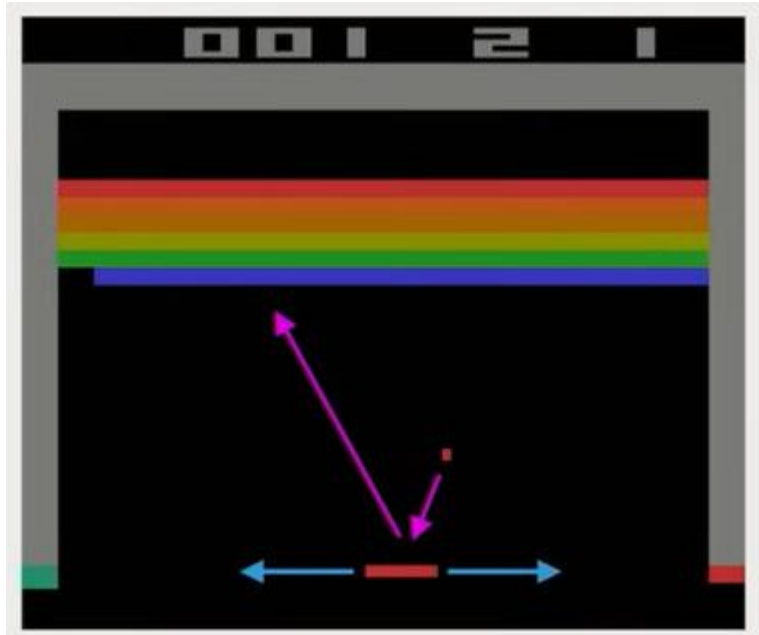




Cel: Maksymalizacja wyniku gry
Stan: Surowe piksele ekranu (210x160)
Akcja: W lewo, w prawo, w górę, w dół
Nagroda: Wzrost/spadek wyniku w momencie t

<https://openai.com/index/gym-retro/>

Geneza



Playing Atari with Deep Reinforcement Learning

Volodymyr Mnih Koray Kavukcuoglu David Silver Alex Graves Ioannis Antonoglou

Daan Wierstra Martin Riedmiller

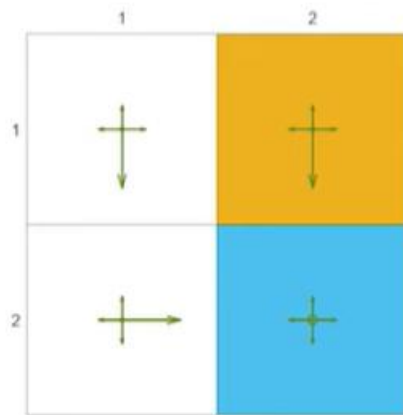
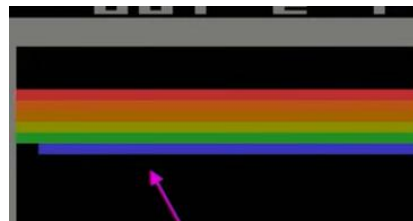
DeepMind Technologies

DOI: 10.1038/nature14236 • Corpus ID: 205242740

Human-level control through deep reinforcement learning

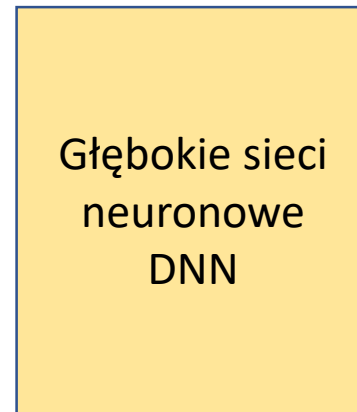
[Volodymyr Mnih](#), [K. Kavukcuoglu](#), [+16 authors](#) [D. Hassabis](#) • Published in [Nature](#) 25 February 2015 • Computer Science

stan+akcja → oczekiwana nagroda



stan, s, akcja a

input



Głębokie sieci
neuronowe
DNN

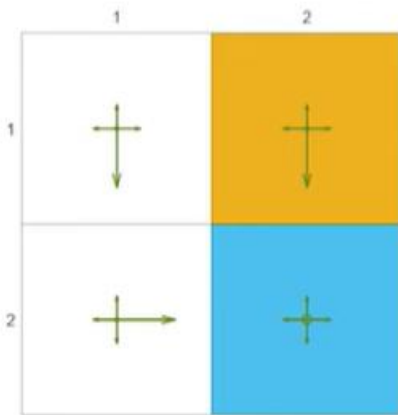
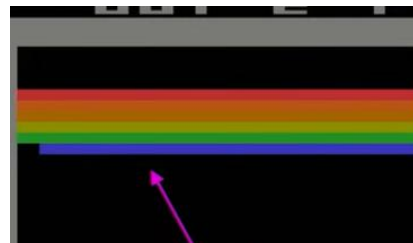
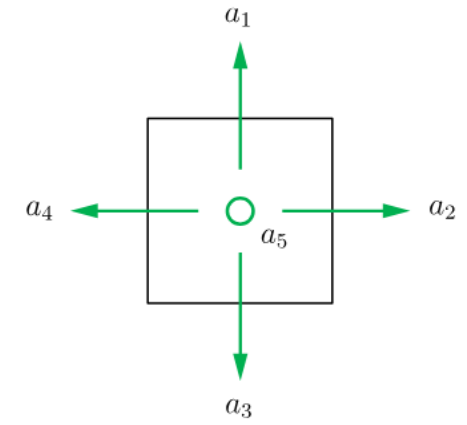
agent



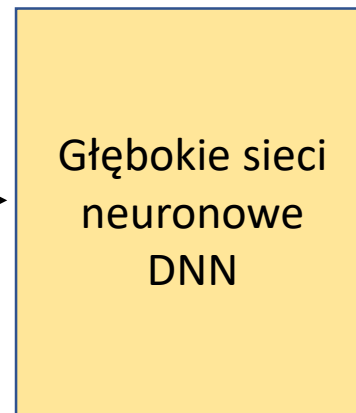
$q(s, a) = 10$

output

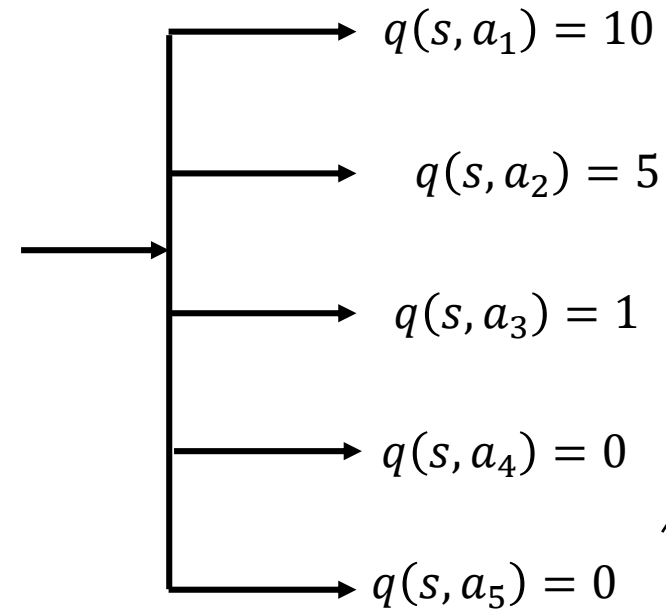
stan $\rightarrow$ oczekiwana nagroda dla kaŹdej akcji



stan, s

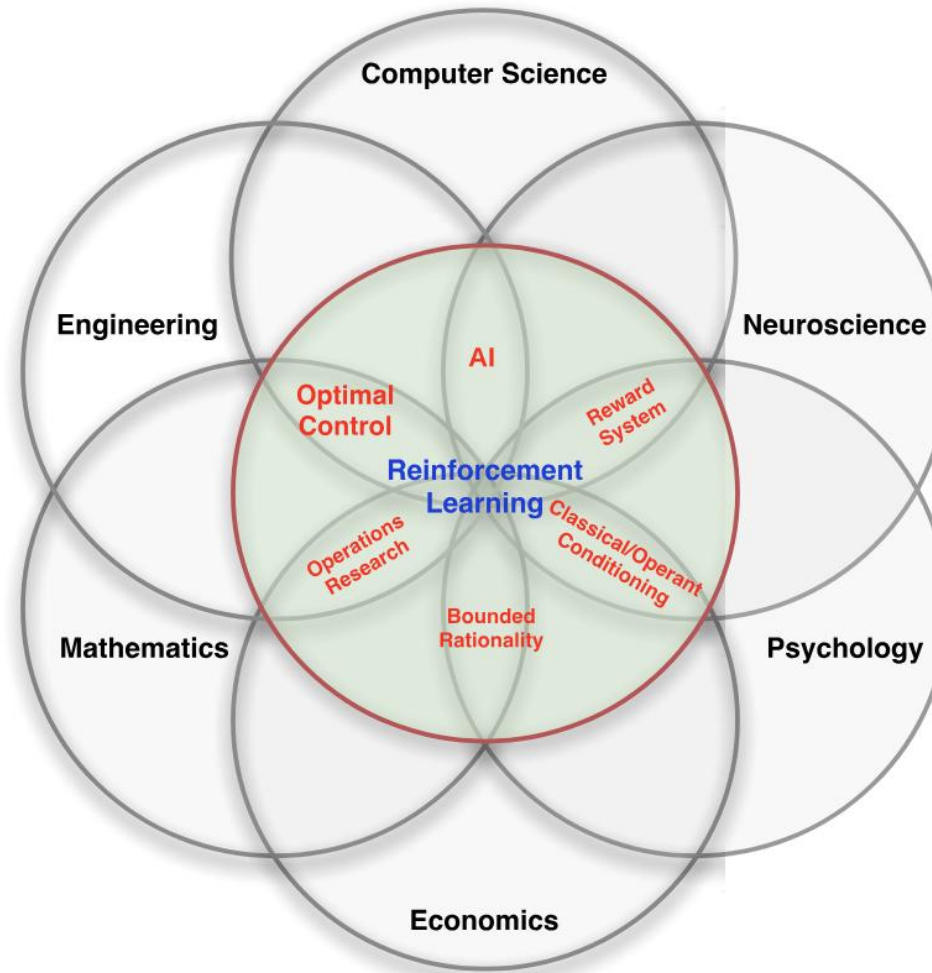


agent

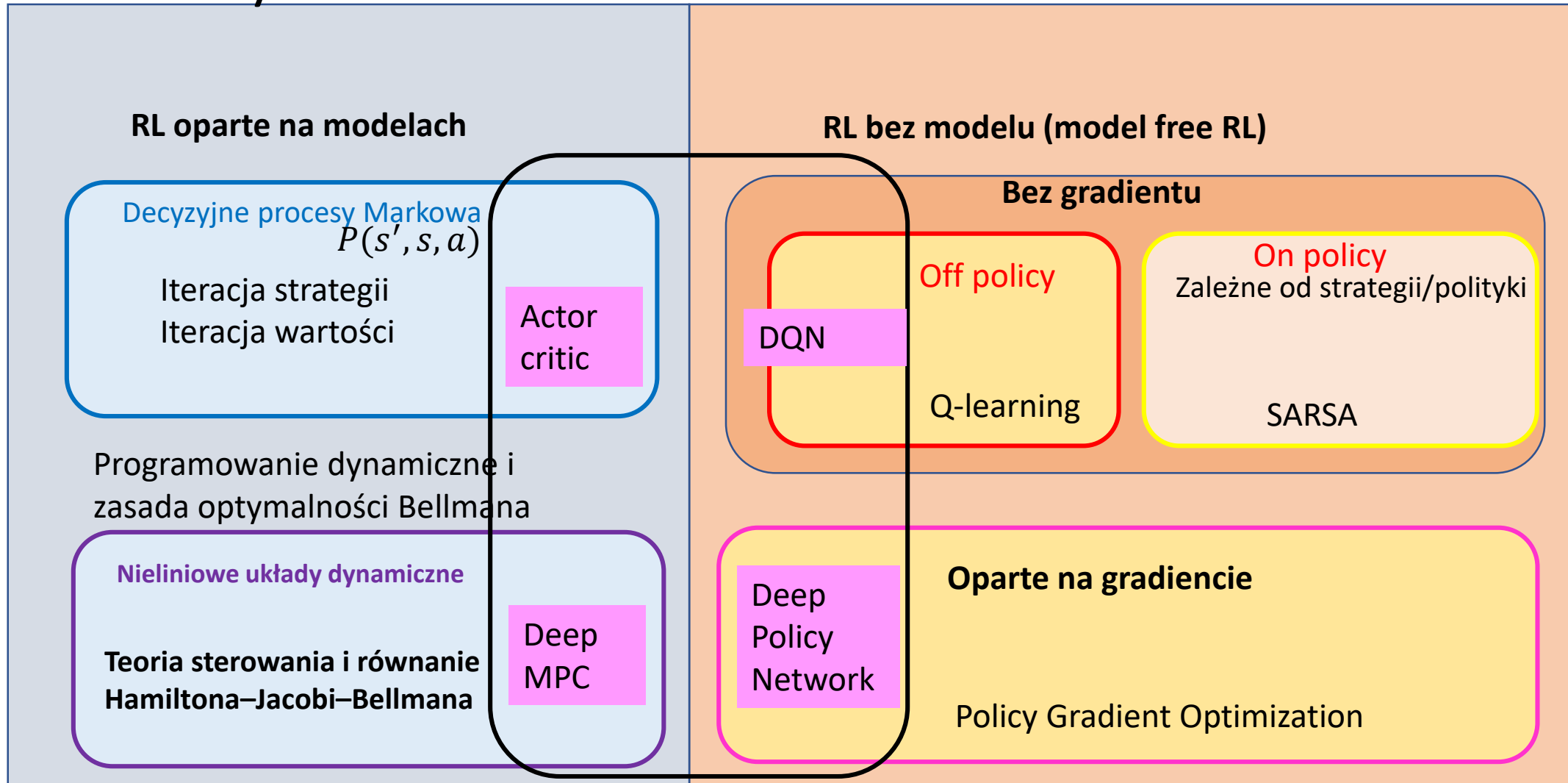


output

$$\pi(s) = \max_a q(s, a) = a_1$$



Metody stosowane w RL



Deep RL

Decyzyjne procesy Markowa (MDP)

Prawie wszystkie zagadnienia uczenia ze wzmocnieniem wykorzystuje metodologię decyzyjnych procesów Markowa (MDP), aby zdefiniować interakcję między uczącym się agentem a jego środowiskiem w kategoriach stanów, akcji/działań i nagród. Pozostałe można sprowadzić do MDP, np. POMDP.

- Sterowanie optymalne- MDP z czasem ciągłym
- bandyci-MDP z jednym stanem

Definicja

Niech S będzie zbiorem skończonym elementów, które nazywamy stanami. Niech wektor $p = [p_i]_{i \in S} = [p(i)]_{i \in S}$ będzie rozkładem prawdopodobieństwa na S . Atomy tego rozkładu nazywają się prawdopodobieństwami początkowymi, zaś wektor p rozkładem początkowym.

Przez $P = [p(ij)]_{i,j \in S} = [p_{ij}]_{i,j \in S}$ oznaczmy macierz stochastyczną, której elementy p_{ij} oznaczają prawdopodobieństwo przejścia ze stanu i do stanu j , $p_{ij} = P(X_{n+1} = j | X_n = i)$.

Macierz $P = [p_{ij}]$ nazywamy macierzą przejścia łańcucha Markowa.

Macierz $P = [p_{ij}]$ jest macierzą stochastyczną, tzn. $\sum_j p_{ij} = 1$.

$$P = \begin{pmatrix} p_{11} & p_{12} & p_{13} \\ p_{21} & p_{22} & p_{23} \\ p_{31} & p_{32} & p_{33} \end{pmatrix}$$

Własność Markowa

Mówimy, że ciąg zmiennych losowych X_n , $n = 0, 1, 2, \dots$, o wartościach w S jest (jednorodnym, skończonym) łańcuchem Markowa, jeżeli spełnione są następujące warunki:

1. $P(X_0 = i) = p(i)$, dla każdego $i \in S$,

2. dla każdego $n \geq 0$ zachodzi równość:

$$P(X_{n+1} = j | X_0 = i_0, \dots, X_n = i) = P(X_{n+1} = j | X_n = i) = p(i, j) = p_{ij}, \text{ dla wszystkich } i_0, \dots, i, j \in S.$$

Andriej Markow

Андрей Андреевич Марков



1856-1922

Własność Markowa

Własność Markowa:

Stan ŁM w chwili $n+1$ zależy wyłącznie od stanu w chwili n .

$$P(X_{n+1} = j | X_0 = i_0, \dots, X_n = i) = P(X_{n+1} = j | X_n = i)$$

Założenie jednorodności:

Prawdopodobieństwa przejścia są niezależne od n : $P(X_{n+1} = j | X_n = i) = p(i, j) = p_{ij}$.

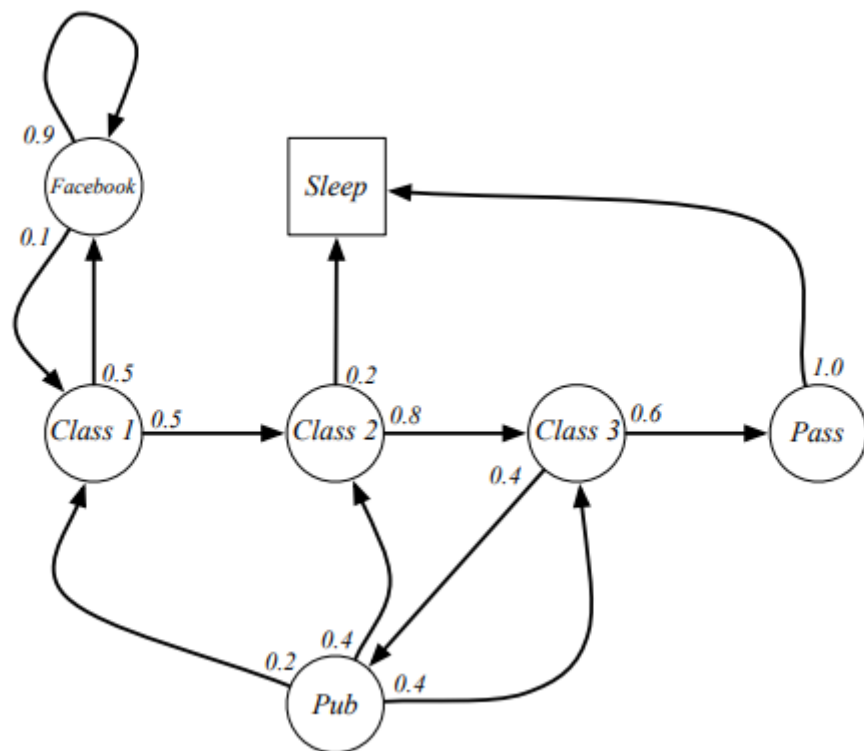
Przyszłość jest niezależna od przeszłości, biorąc pod uwagę teraźniejszość”

Stan przechwytuje wszystkie istotne informacje z historii

Gdy stan jest znany, historia może zostać odrzucona

Stan jest wystarczającą statystyką przyszłości

Macierz przejścia ŁM z wykładu D. Silver



$$\begin{array}{c}
 s \\
 b \\
 p
 \end{array}
 \begin{bmatrix}
 & C1 & C2 & C3 & Pass & Pub & FB & Sleep \\
 & & 0.5 & & & & 0.5 & \\
 & & & 0.8 & & & & 0.2 \\
 & & & & 0.6 & 0.4 & & \\
 0.2 & & & & & & & 1.0 \\
 0.1 & 0.2 & 0.4 & 0.4 & & & 0.9 & \\
 & & & & & & & 1
 \end{bmatrix}$$

Proces Markowa z nagrodami/wartościami

Definicja

Proces Markowa z nagrodami to krotka $\langle S, P, R, \gamma \rangle$, gdzie S -skończony zbiór stanów;

P -macierz migracji (transition matrix),

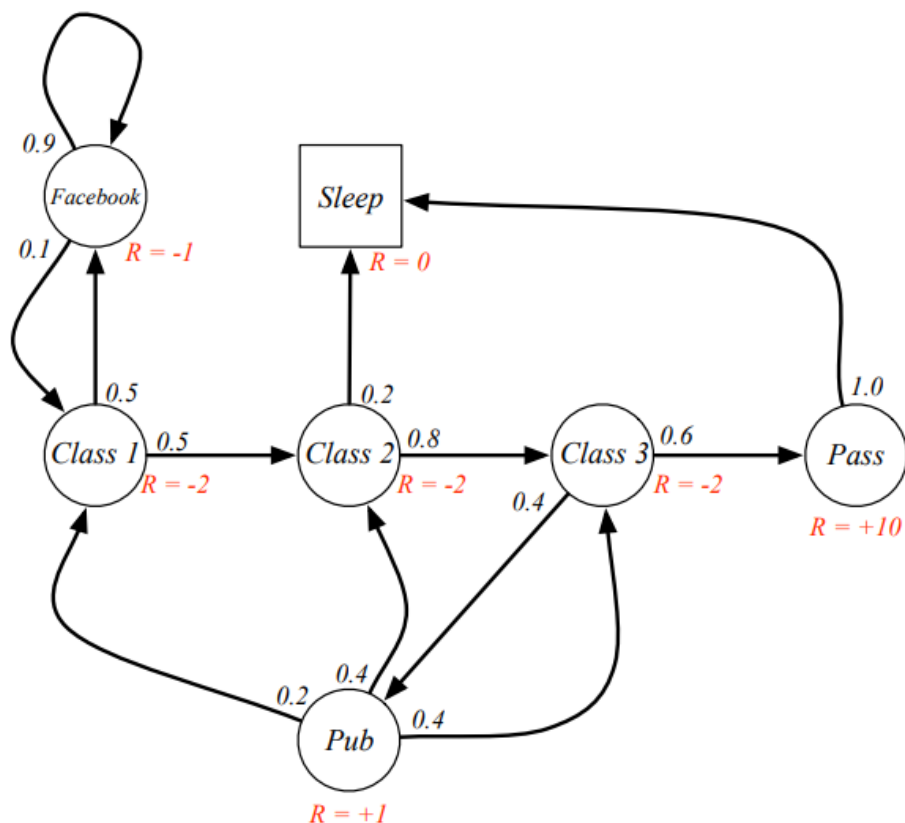
$$p_{ss'} = P(X_{t+1} = s' | X_t = s).$$

Funkcja nagrody R_s

$R_s = E(R_{t+1} | X_t = s)$, $\gamma \in [0,1)$, czynnik dyskontujący

Łańcuch (proces) Markowa z wartościami

Niech $r(s)$ oznacza nagrodę natychmiastową otrzymywaną w stanie s .



G_t – oczekiwana (przyszła) nagroda w chwili t

$$G_t = R_{t+1} + R_{t+2} + \dots = \sum_{i=t+1}^{\infty} R_i$$

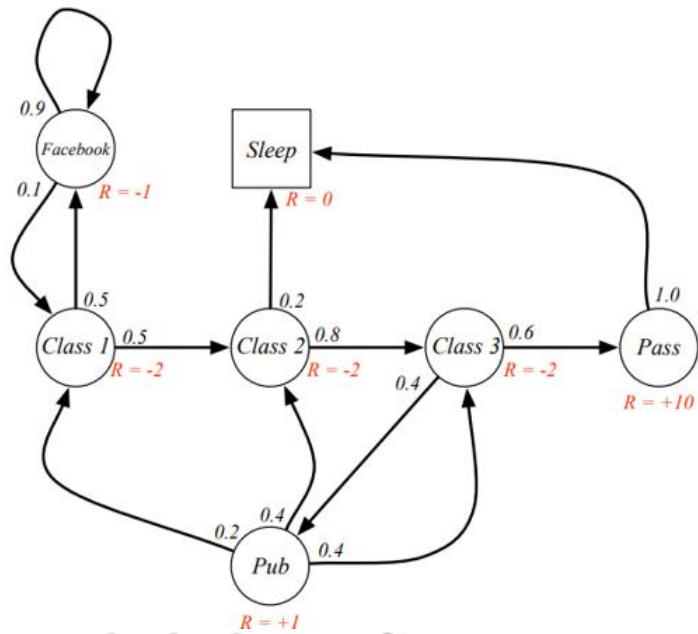
Zdyskontowana oczekiwana nagroda:

$R_t = \sum_{i=t+1}^{\infty} \gamma^i R_i$, gdzie $\gamma \in [0,1)$, discount factor

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

G_t i R_t są zmiennymi losowymi

Użyteczności stanu C1 z gamma=0.5



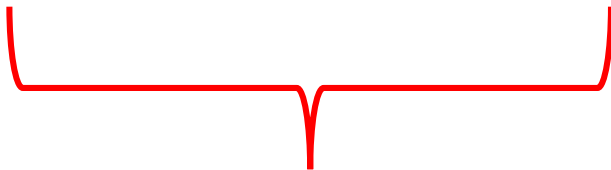
C1 C2 C3 Pass Sleep

C1 FB FB C1 C2 Sleep

C1 C2 C3 Pub C2 C3 Pass Sleep

C1 FB FB C1 C2 C3 Pub C1 ...

FB FB FB C1 C2 C3 Pub C2 Sleep



$$G_t = R_{t+1} + \gamma R_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

$$v_1 = -2 - 2 * \frac{1}{2} - 2 * \frac{1}{4} + 10 * \frac{1}{8} = -2.25$$

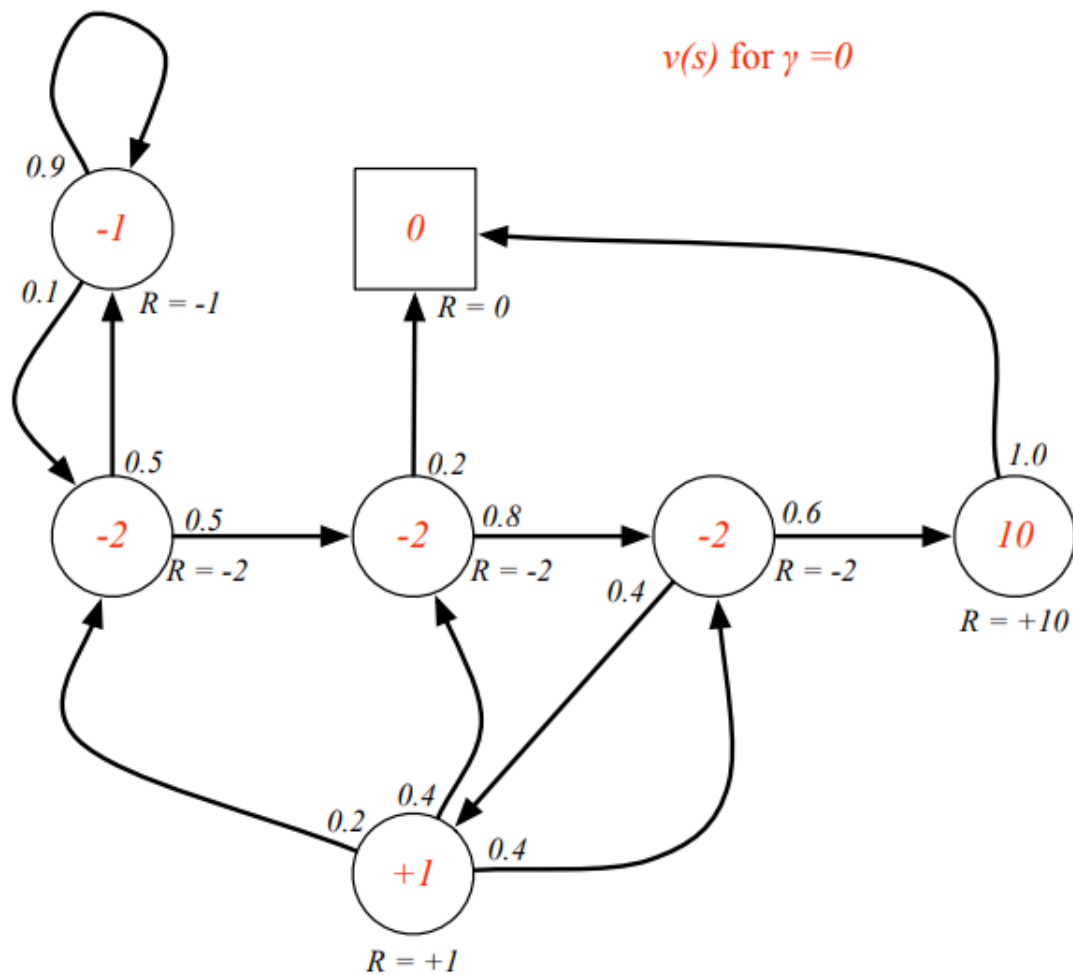
$$v_1 = -2 - 1 * \frac{1}{2} - 1 * \frac{1}{4} - 2 * \frac{1}{8} - 2 * \frac{1}{16} = -3.125$$

$$v_1 = -2 - 2 * \frac{1}{2} - 2 * \frac{1}{4} + 1 * \frac{1}{8} - 2 * \frac{1}{16} \dots = -3.41$$

$$v_1 = -2 - 1 * \frac{1}{2} - 1 * \frac{1}{4} - 2 * \frac{1}{8} - 2 * \frac{1}{16} \dots = -3.20$$

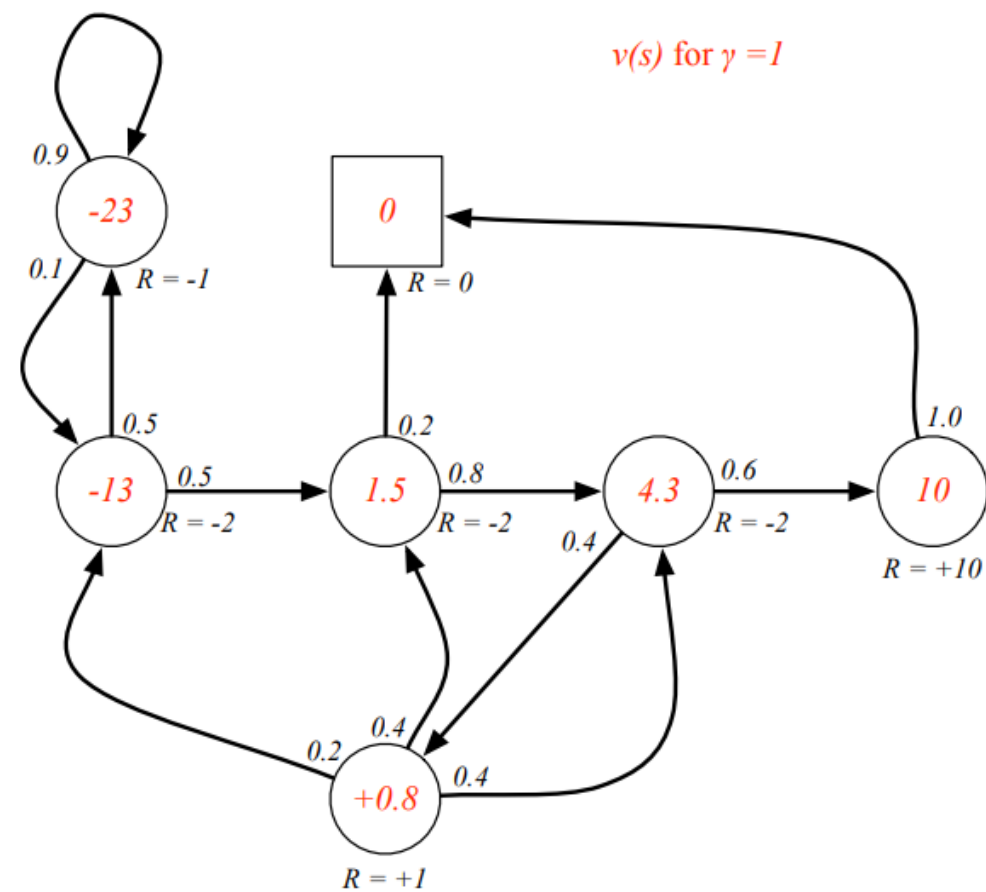
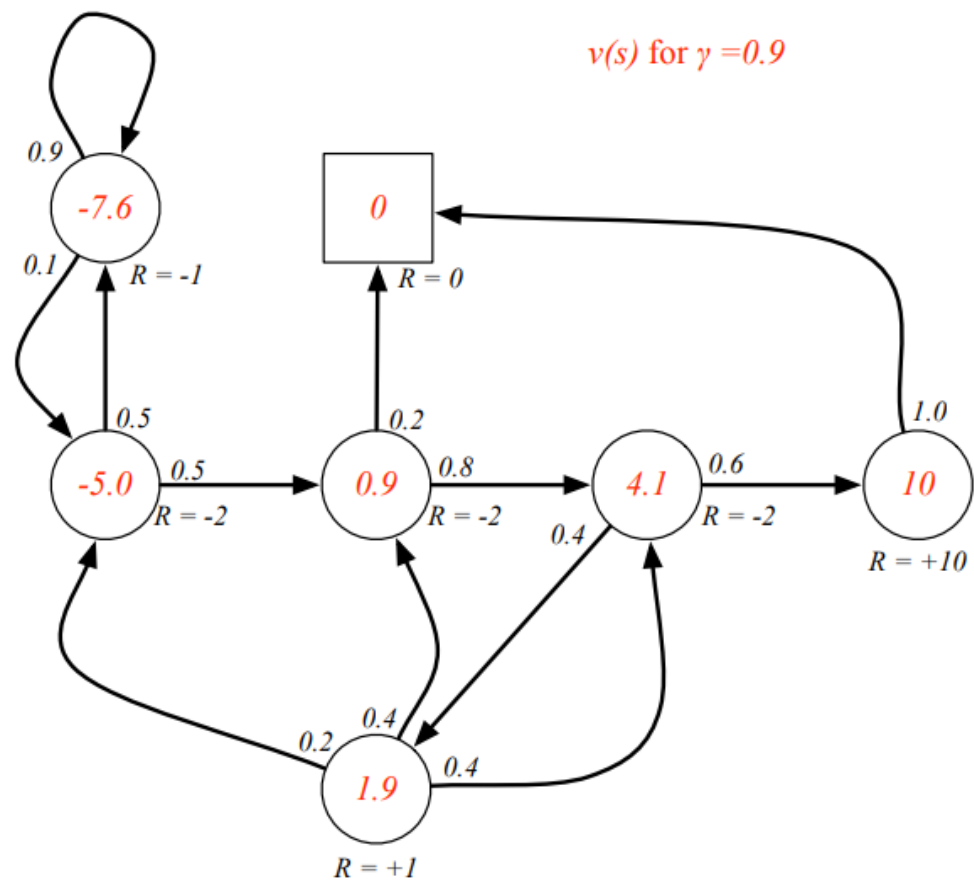
$$\begin{aligned} G_t &\doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \dots \\ &= R_{t+1} + \gamma (R_{t+2} + \gamma R_{t+3} + \gamma^2 R_{t+4} + \dots) \\ &= R_{t+1} + \gamma G_{t+1} \end{aligned}$$

Epizody, próby o skończonej liczbie kroków, trajektorie ŁM



Funkcja wartości stanu $v(s)$ procesu Markowa z nagrodami jest oczekiwaną nagrodą /użytecznością z bycia w stanie s .

$$v(s) = \mathbb{E}[G_t \mid S_t = s]$$



Funkcja wartości stanu

$$\begin{aligned}v(s) &= \mathbb{E}[G_t \mid S_t = s] \\&= \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s] \\&= \mathbb{E}[R_{t+1} + \gamma (R_{t+2} + \gamma R_{t+3} + \dots) \mid S_t = s] \\&= \mathbb{E}[R_{t+1} + \gamma G_{t+1} \mid S_t = s] \\&= \mathbb{E}[R_{t+1} + \gamma v(S_{t+1}) \mid S_t = s]\end{aligned}$$

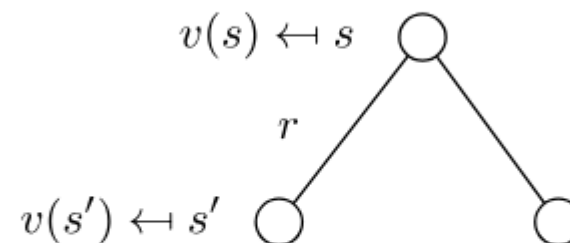
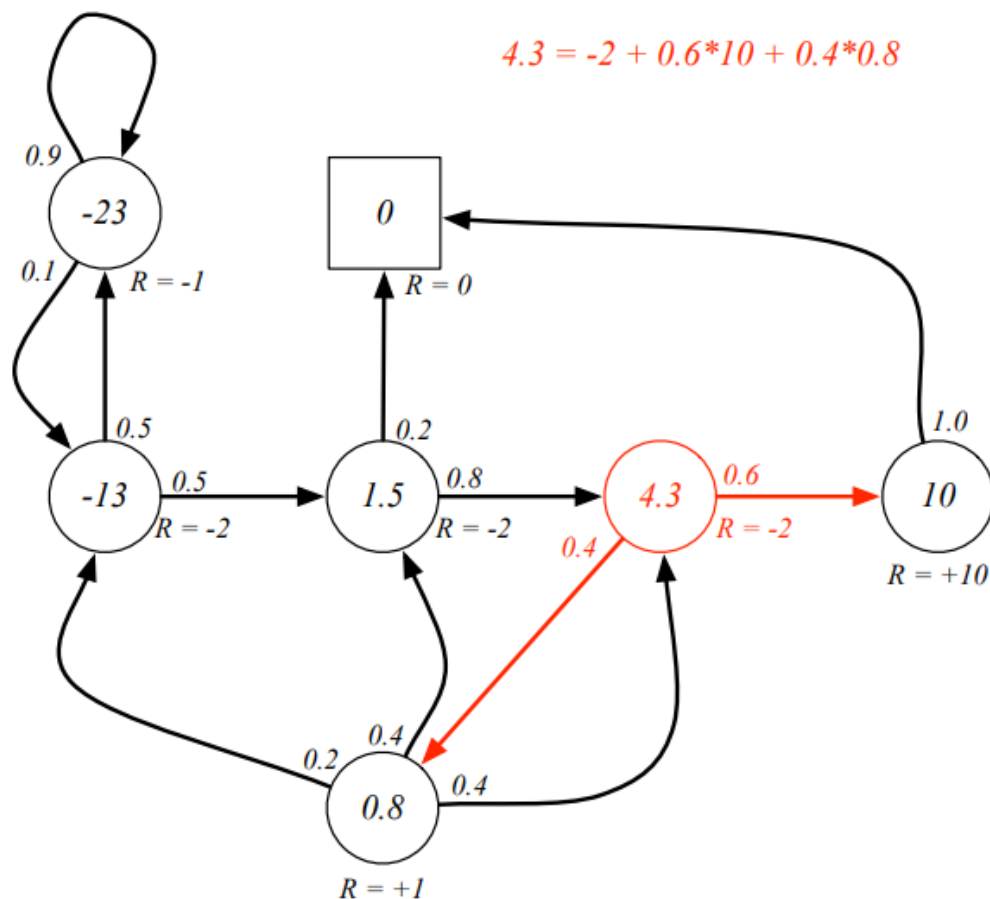
$$v(s) = \mathbb{E}[G_t \mid S_t = s]$$

Funkcja wartości stanu rozkłada się na 2 części

- Nagrodę natychmiastową R_{t+1}
- Zdyskontowaną wartość stanu następnego

Równanie Bellmana

$$v(s) = \mathbb{E}[R_{t+1} + \gamma v(S_{t+1}) \mid S_t = s]$$



$$v(s) = \mathcal{R}_s + \gamma \sum_{s' \in \mathcal{S}} P_{ss'} v(s')$$

Funkcja wartości stanu $V^\pi(s)$ (wartość stanu)

$V^\pi(s)$ zależy od s oraz π , ale nie zależy od czasu t .

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

$$= R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \dots)$$

$$= R_{t+1} + \gamma G_{t+1},$$

$$G_{t+1} = R_{t+2} + \gamma R_{t+3} + \dots$$

$$V^\pi(s) = v_\pi(s) = \mathbb{E}[G_t | S_t = s]$$

$$= \mathbb{E}[R_{t+1} + \gamma G_{t+1} | S_t = s]$$

$$= \mathbb{E}[R_{t+1} | S_t = s] + \gamma \mathbb{E}[G_{t+1} | S_t = s].$$

Procesy Decyzyjne Markowa

MDP to ŁM z nagrodami, w którym wprowadzamy decyzje/akcje.

Jest to środowisko, w którym wszystkie stany mają własność Markowa.

$$M = \{S, A, P, R, \gamma\}$$

Zbiory:

S – przestrzeń stanów;

A – przestrzeń działań, akcji, $A(s)$ -zbiór działań dla stanu $s \in S$

R – zbiór nagród; $R(s, a)$ -nagrody dla każdej pary (s, a)

(lub $r(s, a, s')$ nagroda natychmiastowa za migrację z s do s' za pomocą działania a)

Proces Decyzyjny Markowa

Model/dynamika

P—operator migracji/przejścia (state transition operator, tensor)

$$P(s, a, s') = p(s'|s, a)$$

$$\sum_{s' \in S} p(s'|s, a) = 1 \text{ dla każdej pary } (s, a)$$

Prawdopodobieństwo nagrody r , dla stanu s i akcji a : $p(r|s, a)$

$$\sum_{r \in R(s, a)} p(r|s, a) = 1$$

Wartość stanu (state value function V, v)

Funkcja wartości stanu $V_\pi(s)$ - jak dobrze być w stanie s , działając zgodnie ze strategią π .

Szacuje przyszłe skumulowane nagrody, które agent otrzyma z danego stanu.

Oczekiwana suma nagród, jaką możemy otrzymać, rozpoczynając w stanie s i działając zgodnie ze strategią π .

$$V_\pi(s) = E(G_t | X_t = s)$$

G_t i R_t są zmiennymi losowymi

Strategia/Policy

Strategia, to odwzorowanie $\pi: S \rightarrow A$, z przestrzeni stanów na przestrzeń akcji.

Prawdopodobieństwo wyboru akcji a w stanie s jest równe $\pi(a|s)$

$$\pi(a|s) = P(A_t = a | X_t = s)$$

$$\sum_{a \in A(s)} \pi(a|s) = 1$$

$$\mathcal{P}_{s,s'}^\pi = \sum_{a \in \mathcal{A}} \pi(a|s) \mathcal{P}_{ss'}^a$$

$$\mathcal{R}_s^\pi = \sum_{a \in \mathcal{A}} \pi(a|s) \mathcal{R}_s^a$$

Strategia mówi agentowi jaką akcję ma wybrać w każdym stanie.

Strategia deterministyczna: $a = \pi(s)$

Strategia stochastyczna: $\pi(a|s) = P(a_t = a | X_t = s)$

Własność Markowa

Własność Markowa dotyczy braku pamięci dla procesu dotyczącego kolejnych nagród oraz procesu dotyczącego zmiany stanu.

Mamy następujące trajektorie (epizody):

$s_0, a_0, r_1, s_1, a_1, r_2, s_2, \dots, r_T, s_T$, s_T -stan terminalny

$$p(s_{t+1}|s_t, a_t, s_{t-1}, \dots, s_0, a_0) = p(s_{t+1}|s_t, a_t)$$

$$p(r_{t+1}|s_t, a_t, s_{t-1}, \dots, s_0, a_0) = p(r_{t+1}|s_t, a_t)$$

Ze względu na własność Markowa, MDP może być całkowicie opisany przez:

- **Funkcję nagrody** $r : S \times A \rightarrow \mathbb{R}$

$r_a(s)$ = **natychmiastowa** nagroda, jeśli agent jest w stanie s i podejmuje działanie a . Jest to nagroda **krótkoterminowa**.

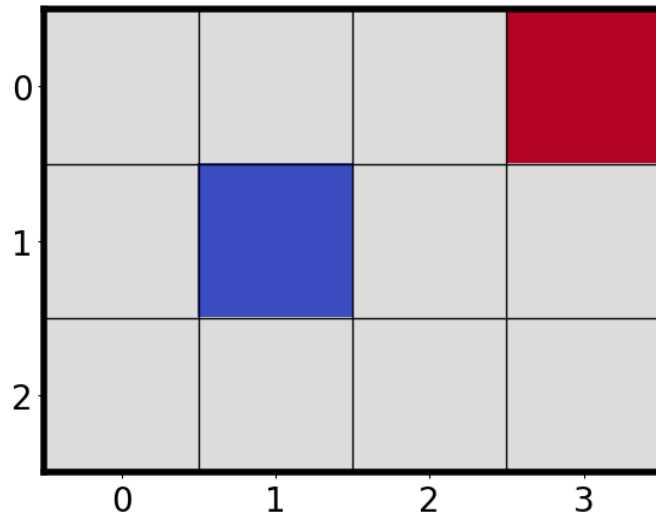
- **Model przejścia** (dynamika procesu):

$$P: S \times A \times S \rightarrow [0, 1]$$

prawdopodobieństwo przejścia z s do s' pod warunkiem podjęcia akcji/działania a

$$P(s, a, s') = p(s'|s, a)$$

Przykład



```
# Move right:
elif state_x < n_x-1 and state_x > 0:
    next_state = (state_y, state_x + 1)
elif state_x == 0 and state_y == 0:
    next_state = (state_y, state_x + 1)

# Move up:
elif state_x == n_x-1:
    next_state = (state_y - 1, state_x)
elif state_x == 0 and state_y > 0:
    next_state = (state_y - 1, state_x)

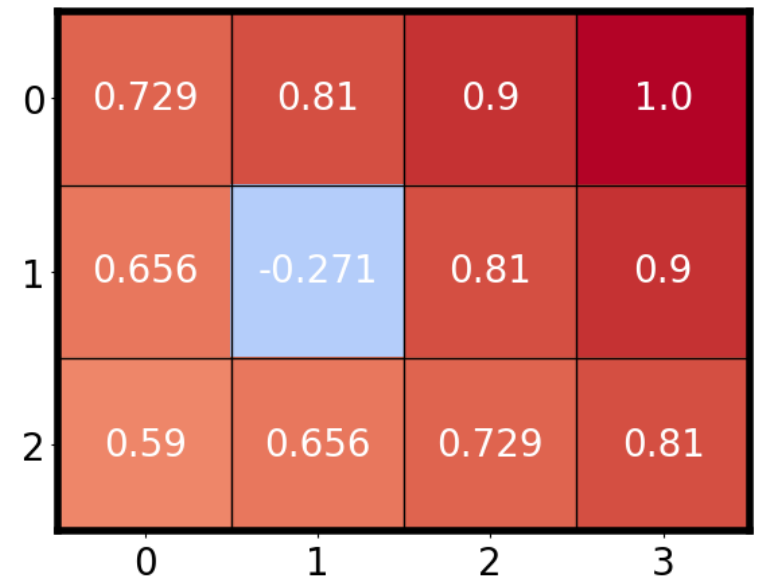
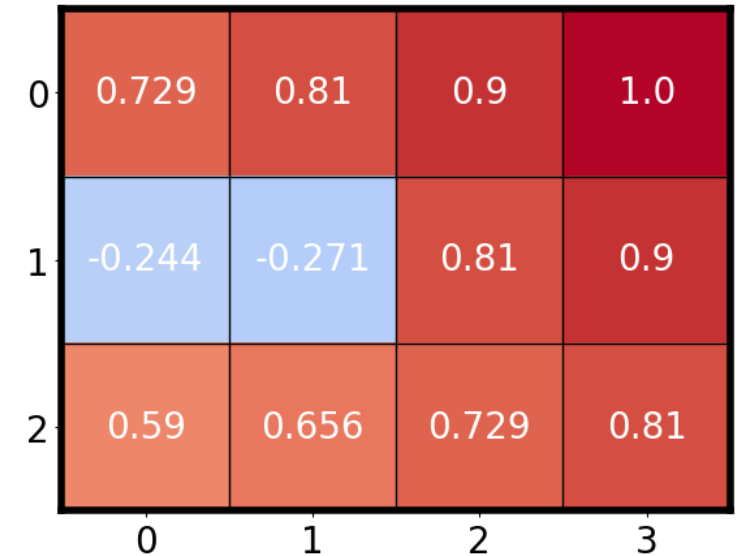
return next_state
```

- Strategia 1

Dla wszystkich stanów z wyjątkiem ostatniej kolumny idziemy w prawo. W ostatniej kolumnie w górę.

- Strategia 2

Omijamy stan zabroniony



Cel: Znalezienie strategii (maksymalizującej nagrodę)

Strategia to może być sieć neuronowa, albo tablica dla dyskretnej przestrzeni stanów i akcji.

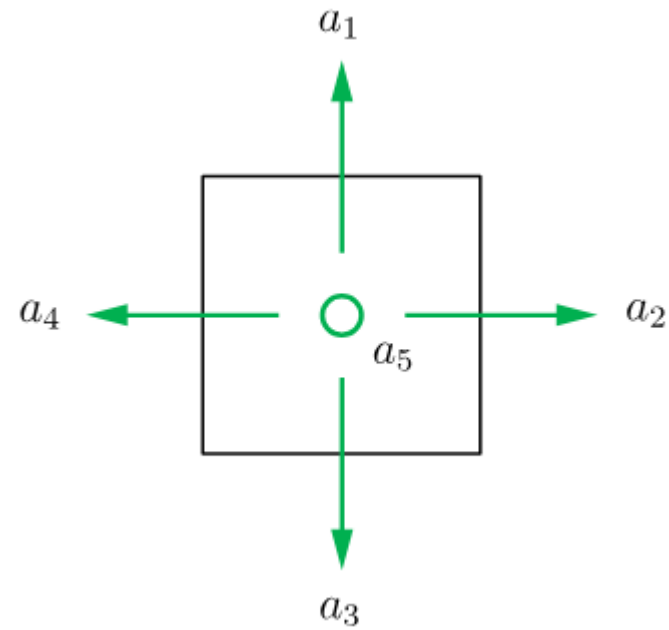
Nie zawsze dany jest model.

Dla danego modelu mamy 2 algorytmy:

- Iteracja wartości
- Iteracja strategii

Przykład

s_1	s_2	s_3
s_4	s_5	s_6
s_7	s_8	s_9

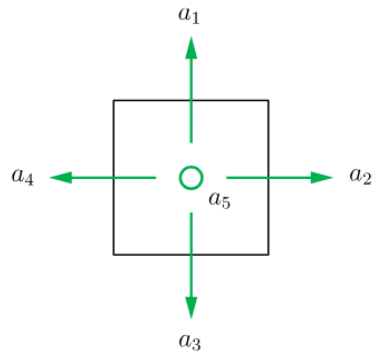


5 możliwych akcji.

Przykład środowiska na siatce. Mamy 9 stanów. Pomarańczowe stany są zabronione (forbidden).

Przykład

s_1	s_2	s_3
s_4	s_5	s_6
s_7	s_8	s_9



$$p(s_1|s_1, a_2) = 0$$

$$p(s_2|s_1, a_2) = 1$$

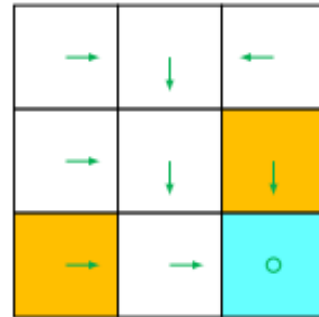
$$p(s_3|s_1, a_2) = 0$$

$$p(s_4|s_1, a_2) = 0$$

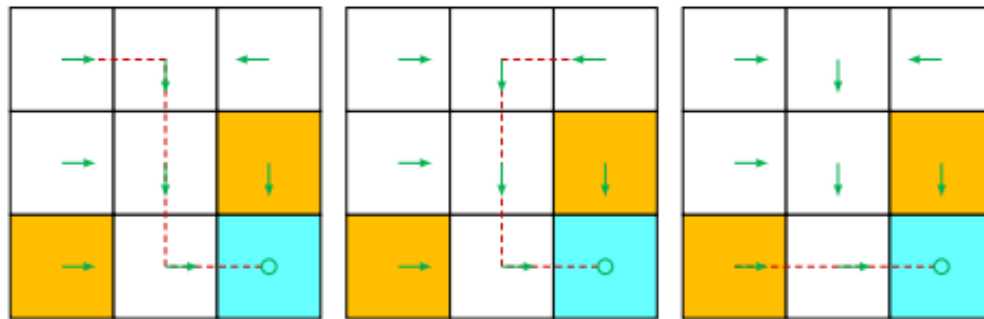
$$p(s_5|s_1, a_2) = 0$$

Działanie i Strategia

Deterministyczna strategia



(a) A deterministic policy



(b) Trajectories obtained from the policy

$$\pi(a_1|s_1) = 0$$

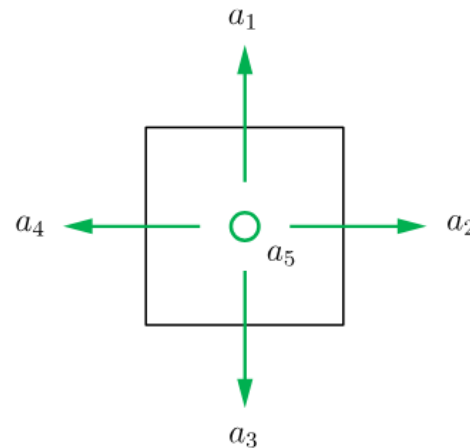
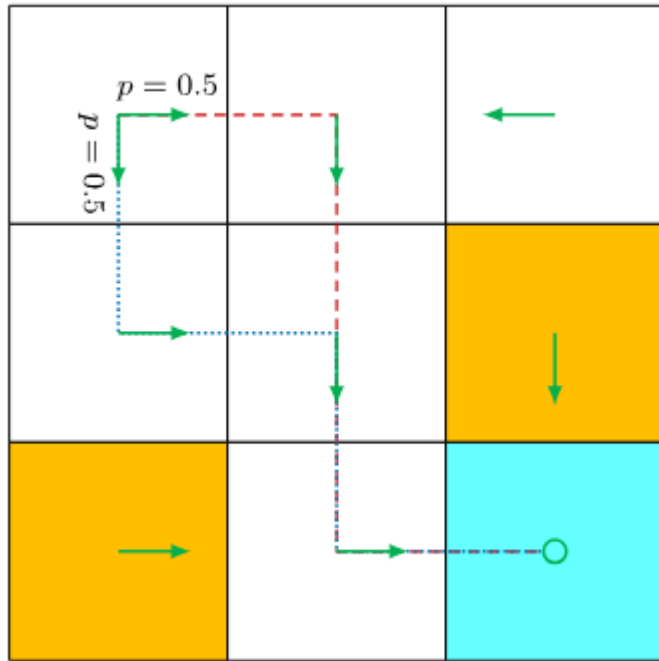
$$\pi(a_2|s_1) = 1$$

$$\pi(a_3|s_1) = 0$$

$$\pi(a_4|s_1) = 0$$

Strategia/polityka mówi agentowi jaką akcję ma wybrać w każdym stanie.

Przykład strategii stochastycznej



$$\pi(a_1|s_1) = 0$$

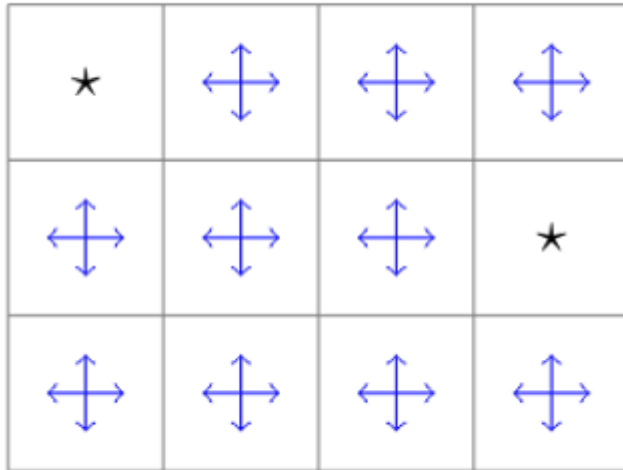
$$\pi(a_2|s_1) = 0.5$$

$$\pi(a_3|s_1) = 0.5$$

$$\pi(a_4|s_1) = 0$$

$$\pi(a_5|s_1) = 0$$

Przykład



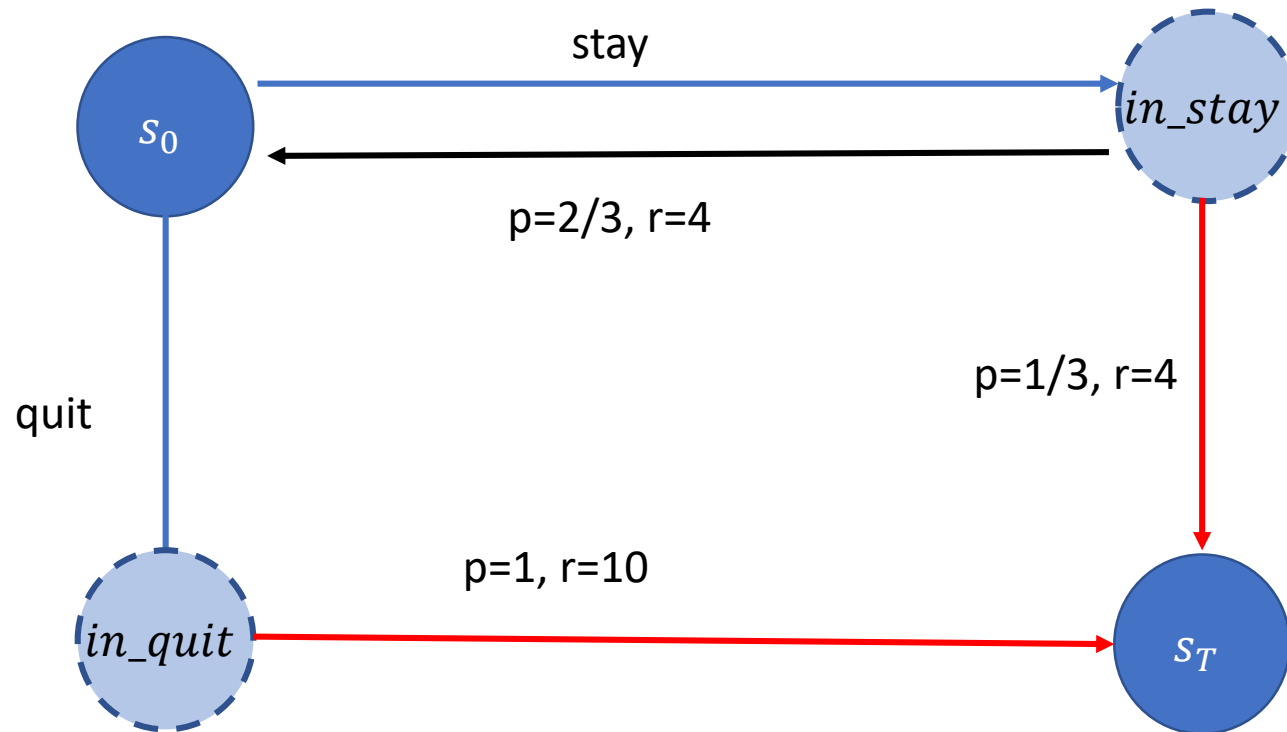
Akcje:

1. Iść w prawo →
2. Iść w lewo ←
3. Iść w dół ↓
4. Iść w górę ↑

Nagroda: -1 za każde przejście

Cel: dotarcie do stanu terminalnego (*) w najmniejszej liczbie kroków

Przykład-rzut kostką



$A = \{\text{stay, quit}\}$

Po decyzji „stay” rzucamy kostką.

Jeśli wypadnie 1 lub 2 oczka przechodzimy do s_T .

W przeciwnym razie kontynuujemy-wracamy do s_0 .

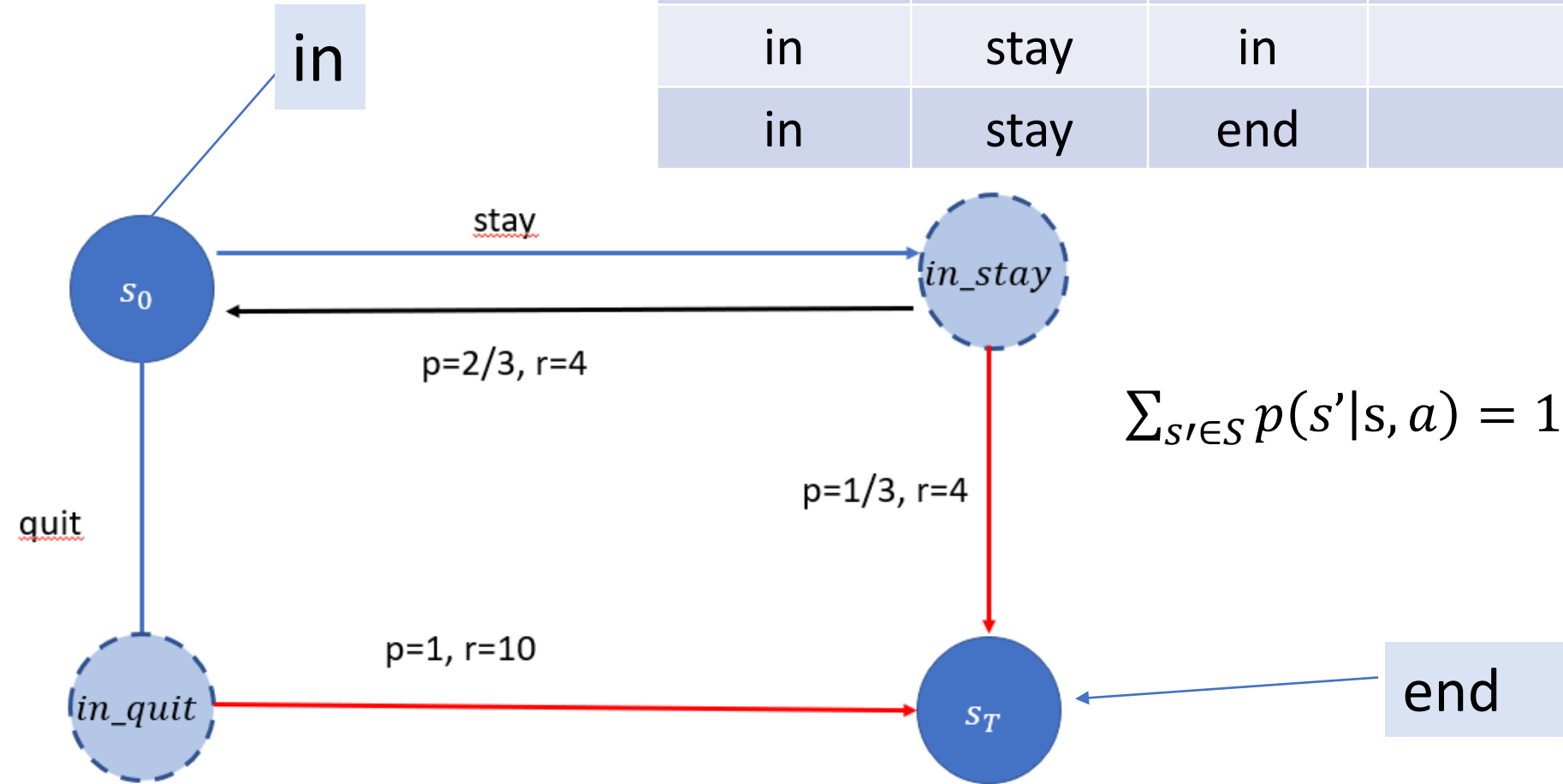
$$\gamma = 1$$

s_T -stan terminalny, koniec gry

Jeśli rezygnujemy z gry, otrzymujemy $r=10$.

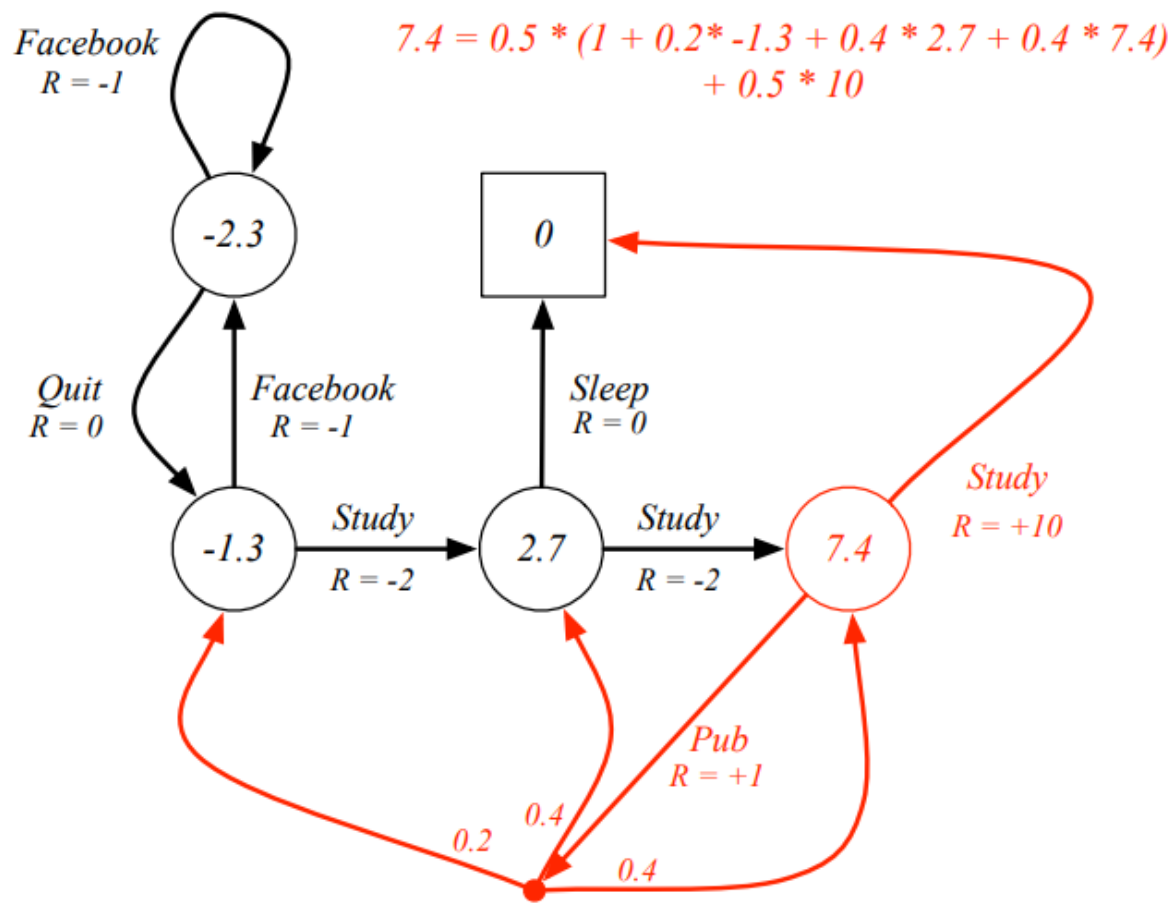
Jeśli zostajemy, otrzymujemy $r=4$ i rzucamy kostką.

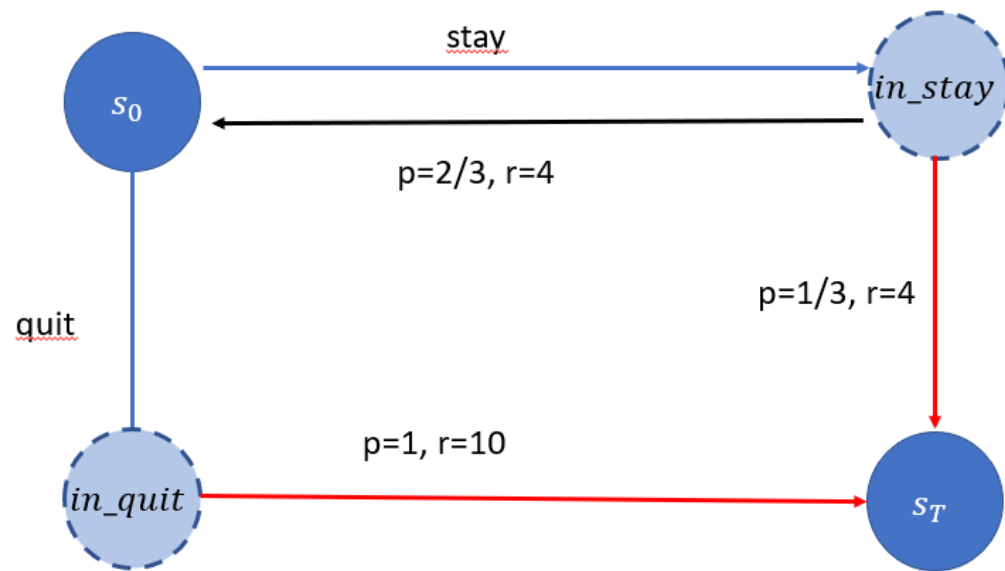
s	a	s'	$P(s, a, s') = p(s' s, a)$
in	quit	end	1
in	stay	in	2/3
in	stay	end	1/3



$$\sum_{s' \in S} p(s'|s, a) = 1 \text{ dla ka\k{z}dej pary } (s, a)$$

Przykład





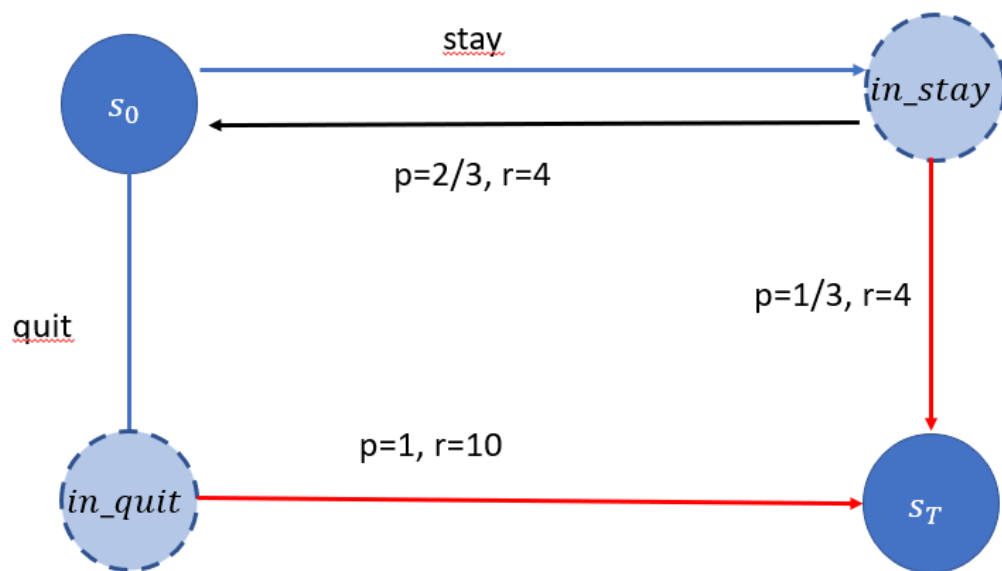
Trajektoria

Wartość

- $[s_0, stay, 4, end]$ 4
- $[s_0, stay, 4, s_0, stay, 4, s_0, stay, 4, end]$ 12
- $[s_0, stay, 4, s_0, stay, 4, end]$ 8
- $[s_0, stay, 4, s_0, stay, 4, s_0, stay, 4, s_0, stay, 4, end]$ 16

Związek między $V_\pi(s)$ i $Q_\pi(s, a)$

- $V_\pi(s) = \begin{cases} 0 & \text{dla } s_T \\ Q_\pi(s, \pi(s)) & \text{dla pozostałych stanów} \end{cases}$



$V_\pi(s)$ dla $\pi(s_0) = stay$

$$V_\pi(s_0) = Q_\pi(s_0, stay)$$

$$V_\pi(s_T) = Q_\pi(s_0, stay)$$

$$V_\pi(s_T) = 0$$

$$V_\pi(s_0) = 1/3(4 + V_\pi(s_T)) + 2/3(4 + V_\pi(s_0))$$

$$V_\pi(s_0) = 12$$

Funkcja wartości stanu $V^\pi(s)$ (wartość stanu)

$$\begin{aligned}v_\pi(s) &= \mathbb{E}[G_t | S_t = s] \\&= \mathbb{E}[R_{t+1} + \gamma G_{t+1} | S_t = s] \\&= \mathbb{E}[R_{t+1} | S_t = s] + \gamma \mathbb{E}[G_{t+1} | S_t = s].\end{aligned}$$

Wartość oczekiwana nagrody natychmiastowej

$$\begin{aligned}\mathbb{E}[R_{t+1} | S_t = s] &= \sum_{a \in \mathcal{A}} \pi(a|s) \mathbb{E}[R_{t+1} | S_t = s, A_t = a] \\&= \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{r \in \mathcal{R}} p(r|s, a) r.\end{aligned}$$

Funkcja wartości stanu $V^\pi(s)$ (wartość stanu)

Wartość oczekiwana nagrody przyszłej

$$\begin{aligned}\mathbb{E}[G_{t+1}|S_t = s] &= \sum_{s' \in \mathcal{S}} \mathbb{E}[G_{t+1}|S_t = s, S_{t+1} = s']p(s'|s) \\ &= \sum_{s' \in \mathcal{S}} \mathbb{E}[G_{t+1}|S_{t+1} = s']p(s'|s) \quad \text{Własność Markowa} \\ &= \sum_{s' \in \mathcal{S}} v_\pi(s')p(s'|s) \\ &= \sum_{s' \in \mathcal{S}} v_\pi(s') \sum_{a \in \mathcal{A}} p(s'|s, a)\pi(a|s).\end{aligned}$$

Funkcja wartości stanu $V^\pi(s)$ (wartość stanu)

$$\begin{aligned} v_\pi(s) &= \mathbb{E}[R_{t+1}|S_t = s] + \gamma \mathbb{E}[G_{t+1}|S_t = s], \\ &= \underbrace{\sum_{a \in \mathcal{A}} \pi(a|s) \sum_{r \in \mathcal{R}} p(r|s, a) r}_{\text{oczekiwana nagród natychmiastowych}} + \gamma \underbrace{\sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s' \in \mathcal{S}} p(s'|s, a) v_\pi(s')}_{\text{oczekiwana nagród przyszłych}} \\ &= \sum_{a \in \mathcal{A}} \pi(a|s) \left[\sum_{r \in \mathcal{R}} p(r|s, a) r + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) v_\pi(s') \right] \quad \text{dla wszystkich } s \in \mathcal{S} \end{aligned}$$

Równania Bellmana

$$v_{\pi}(s) = r_{\pi}(s) + \gamma \sum_{s' \in \mathcal{S}} p_{\pi}(s'|s) v_{\pi}(s'),$$

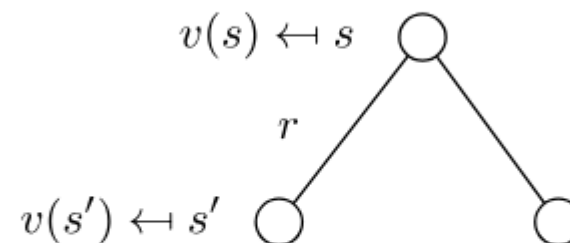
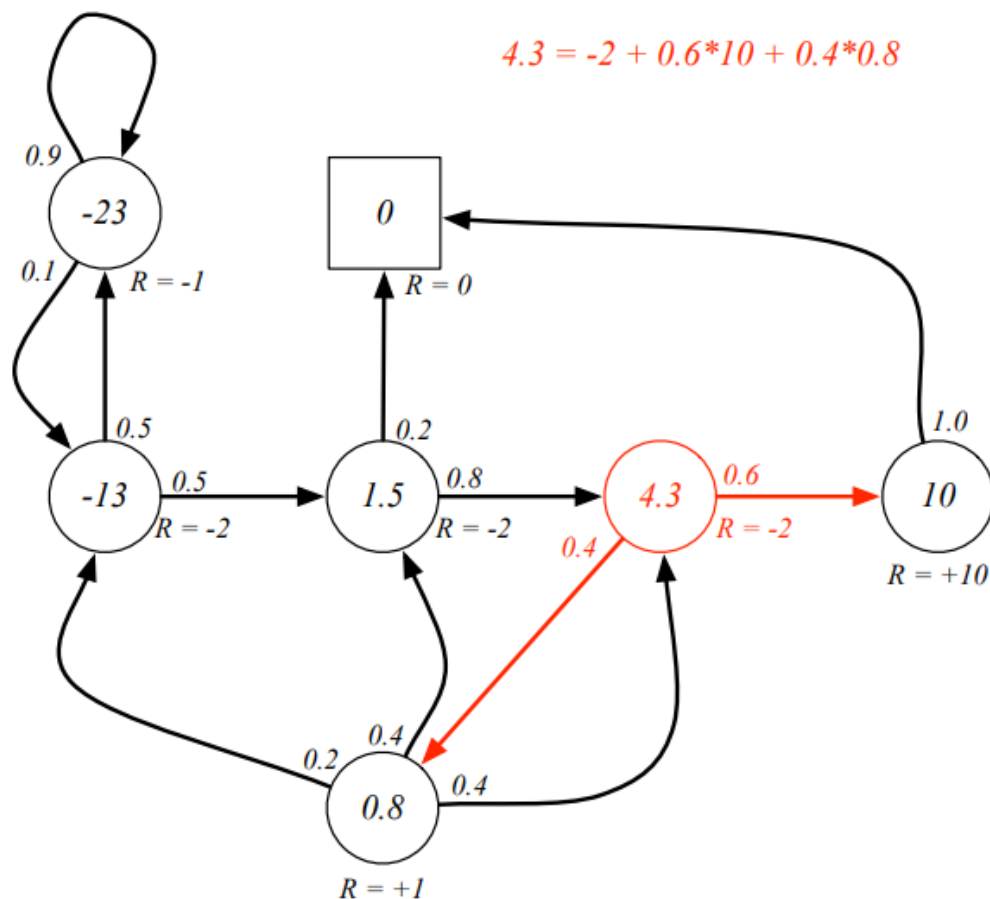
$$r_{\pi}(s) \doteq \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{r \in \mathcal{R}} p(r|s, a) r,$$

$$p_{\pi}(s'|s) \doteq \sum_{a \in \mathcal{A}} \pi(a|s) p(s'|s, a).$$

$$v_{\pi} = r_{\pi} + \gamma P_{\pi} v_{\pi},$$

Równanie Bellmana

$$v(s) = \mathbb{E}[R_{t+1} + \gamma v(S_{t+1}) \mid S_t = s]$$



$$v(s) = \mathcal{R}_s + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'} v(s')$$

Równania Bellmana dla MDP (macierzowe)

- Wyznaczanie wartości stanów z równań Bellmana dla strategii π

$$v_{\pi} = \mathcal{R}^{\pi} + \gamma \mathcal{P}^{\pi} v_{\pi}$$

$$v_{\pi} = (I - \gamma \mathcal{P}^{\pi})^{-1} \mathcal{R}^{\pi}$$

Równania Bellmana oceniają skuteczność strategii poprzez oszacowanie oczekiwanej wartości wynikającej z jej zastosowania.

Równania Bellmana pozwalają wyznaczyć wartości stanów dla konkretnej strategii.

Równania Bellmana dla procesu z nagrodami

$$v = \mathcal{R} + \gamma \mathcal{P}v$$

$$\begin{bmatrix} v(1) \\ \vdots \\ v(n) \end{bmatrix} = \begin{bmatrix} \mathcal{R}_1 \\ \vdots \\ \mathcal{R}_n \end{bmatrix} + \gamma \begin{bmatrix} \mathcal{P}_{11} & \dots & \mathcal{P}_{1n} \\ \vdots & & \\ \mathcal{P}_{n1} & \dots & \mathcal{P}_{nn} \end{bmatrix} \begin{bmatrix} v(1) \\ \vdots \\ v(n) \end{bmatrix}$$

Znany jest głównie jako twórca programowania dynamicznego (Dynamic Programming 1957).

Od lat 40. XX wieku, pomiędzy rokiem 1948 i 1952, Richard Bellman rozwijał teorię programowania dynamicznego. Pracując na wydziale matematyki i w RAND, badał problem określania alokacji pocisków do celów tak aby spowodować możliwie największe szkody. Praca ta doprowadziła go do sformułowania zasady optymalności programowania dynamicznego. W 1957 roku Richard Bellman zastosował programowania dynamicznego do sterowania optymalnego układami dyskretnymi, co ukazało jednocześnie, naturalny zwrot w podejściu do rozwiązywania problemów sterowania.



Rozwiązanie równań Bellmana

$$v = \mathcal{R} + \gamma \mathcal{P}v$$

$$(I - \gamma \mathcal{P})v = \mathcal{R}$$

$$v = (I - \gamma \mathcal{P})^{-1} \mathcal{R}$$

Złożoność obliczeniowa $O(n^3)$, n – liczba stanów

Metody rozwiązywania:

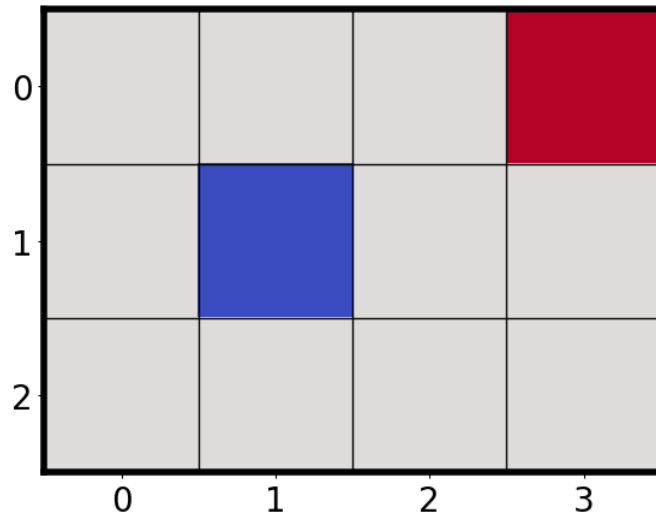
- Dynamiczne;
- Monte Carlo;
- Różnic czasowych (TD)

Richard Ernest Bellman



1920 – 1984

Przykład



```
# Move right:
elif state_x < n_x-1 and state_x > 0:
    next_state = (state_y, state_x + 1)
elif state_x == 0 and state_y == 0:
    next_state = (state_y, state_x + 1)

# Move up:
elif state_x == n_x-1:
    next_state = (state_y - 1, state_x)
elif state_x == 0 and state_y > 0:
    next_state = (state_y - 1, state_x)

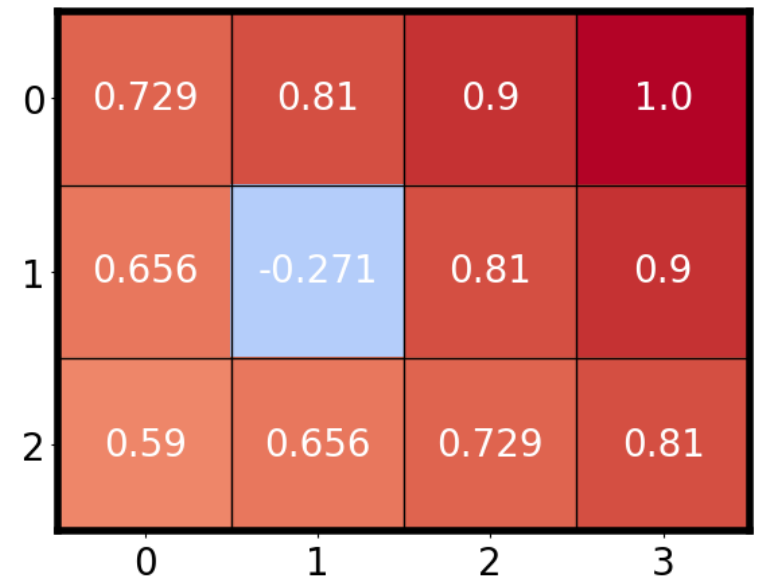
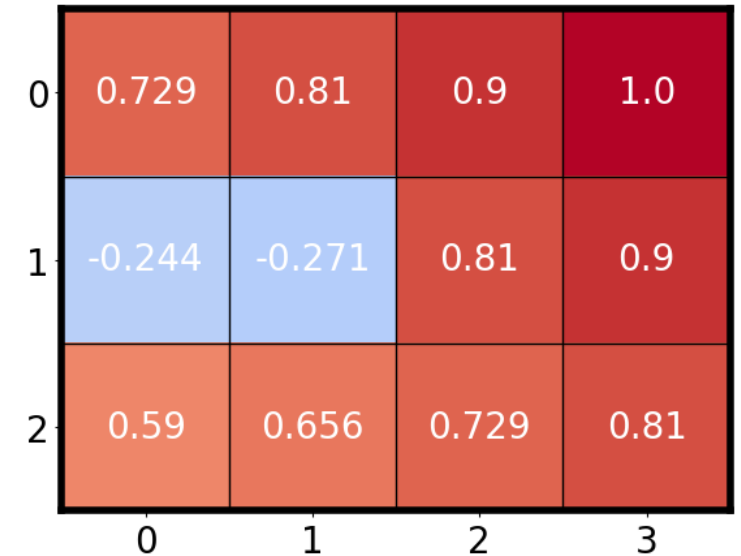
return next_state
```

- Strategia 1

Dla wszystkich stanów z wyjątkiem ostatniej kolumny idziemy w prawo. W ostatniej kolumnie w górę.

- Strategia 2

Omijamy stan zabroniony



Cel: Znalezienie strategii (maksymalizującej nagrodę)

Strategia to może być sieć neuronowa, albo tablica dla dyskretnej przestrzeni stanów i akcji.

Nie zawsze dany jest model.

Strategia optymalna

★	←	↔ ↓	↓
↑	↕	→	★
↑	↕	↗	↑

Strategia optymalna

Pytania

Istnienie: Czy istnieje optymalna strategia?

Jednoznaczność: Czy optymalna strategia jest jedyna?

Stochastyczność: Czy optymalna strategia jest stochastyczna czy deterministyczna?

Algorytm: Jak uzyskać optymalną strategię i optymalne wartości stanu?

Uwaga

- Istnieje co najmniej jedna strategia optymalna.
- Wszystkie strategie optymalne mają tę samą optymalną funkcję wartości stanu V^* i optymalną funkcję wartości akcji Q^* .

Optymalna wartość funkcji stanu i funkcji Q

- $V^*(s) = \max_{\pi} V^{\pi}(s)$ dla każdego stanu $s \in S$
- $Q^*(s, a) = \max_{\pi} Q^{\pi}(s, a)$ dla każdego stanu $s \in S$

Pytania

- Istnienie: Czy to równanie ma rozwiązanie?
- Jednoznaczność: Czy rozwiązanie jest jedyne?
- Algorytm: Jak rozwiązać to równanie?
- Optymalność: W jaki sposób rozwiązanie jest powiązane z optymalną strategią?